poisson-solver Performance Evaluation

Matous Zatloukal

Warning about Data Correctness

Right before the cluster maintenance, on the evening of 4.9.2026, we found out that our `PoisFFT` benchmarks have been incorrectly constructed and some configurations (confirmed that not all) were measured to be slower than they in general are. We believe this slowdown could be due to cache conflict misses but haven't confirmed it. The measured data is correct and `PoisFFT` really behaves this way but it is a special case and won't be observed in most use-cases.

This fact will most likely alter the speedups of `poisson-solver` vs `PoisFFT` (expect smaller speedups) but the analysis and methodology should remain the same.

More details are in the ELMM Local Solvers section.

Another important fact is that the `TridiagZ` (computation kernels for `poisson-solver nonuniform-z` solvers) was accidentally not wrapped in a timer (see benchmark description for more information about timers), thus resulting in a faster kernel time in `nonuniform-z poisson-solver` configurations. This was also fixed and will be remeasured upon the cluster reopening.

AI Usage

LLMs were used for the following tasks:

1. Fortran syntax explanation and navigation of the `ELMM` model,
2. help with rewriting `poisson-solver` (cpp) benchmarks into their `PoisFFT` (Fortran) equivalent,
3. assistance with plotting the data and
4. alternative to a search engine (e.g. "Find sources where I could read about X"),
5. give constructive criticism about this report.

Problem Description

Motivation behind `poisson-solver`

`PoisFFT` is a Fortran library that solves the Poisson equation using the Fast Fourier Transform [1]. It is a part of `ELMM` (Extended Large Eddy Microscale Model), a LES (Large Eddy Simulation) model for atmospheric flows.

`PoisFFT` is written entirely in Fortran and utilizes only CPUs. It is parallelized using `OpenMP` in a single-node (local) setup and using `MPI` in a distributed setup. Furthermore, it offers a hybrid `MPI` + `OpenMP` parallelization in a distributed setup. `PoisFFT` requires the users to first define a domain (number of grid cells, grid lengths, boundary conditions, etc.) and then allows them to repeatedly solve the Poisson equation with this domain configuration by supplying the right-hand side of the equation $\Delta\phi = b$.

Equation solving can be summarized as the following pipeline:

1. Apply a forward FFT on the whole RHS,
2. divide each RHS element by a sum of pre-computed eigenvalues,

3. apply a backward FFT on the whole RHS and
4. normalize the result to get Φ.

For different combinations of BCs (boundary conditions) the pipeline may slightly vary (e.g., the real RHS is first converted into a complex RHS or different types of FFTs are applied separately to 1D pencils in one direction and then to 2D slabs in the other directions). For distributed solvers it is even more complicated due to the requirement of data locality before FFT steps.

Apart from the FFT steps, `PoisFFT` is a 1-point stencil computation which makes it a great candidate for a rewrite into C++ and CUDA to take advantage of the GPUs available on the MFF cluster `chimera`.

`poisson-solver` is exactly this rewrite which we've done as a part of Research Project (NPRG070). As stated before, it utilizes GPUs which are more suited for this type of a problem.

We suspect that the buffer copying between host and device will be the bottleneck in `poisson-solver`. Therefore, `poisson-solver` provides additional API that accepts device buffers. We refer to the APIs as GPU API (no host-device copies) and CPU API (host-device copies).

Local Solvers Evaluation

For non-distributed solvers, we want to determine the speedup of `poisson-solver` compared to `PoisFFT` in all possible combinations of:

1. Precision (double or float),
2. dimensionality of the grid (1, 2 or 3),
3. uniformity of the grid on the Z axis and
   - Both solvers support the option to have the grid cell sizes nonuniform in the Z axis.
4. boundary conditions (dependent on the dimensionality and uniformity of Z axis).

In addition to those, `PoisFFT` will also be measured with different number of cores (16, 32 and 48).

The grid side sizes will range from 64 to 512 with the step of 64.

We also want to do the previous measurements using the GPU API to simulate the situation where other parts of `ELMM` are rewritten and the inputs for `poisson-solver` are already on the device.

To help us determine the bottleneck of the pipeline for further research focus, we will measure the following in `poisson-solver`:

1. Time spent copying buffers,
2. time spent executing FFTs,
3. time spent on our kernel execution (e.g., division by eigenvalues) and
4. total time spent solving the equation.

Only the total time spent solving the equation will be measured for `PoisFFT`. Some metrics are not present (buffer copying) and some would be tedious to incorporate into the code-base as individual pipeline steps are not as cleanly separated as in `poisson-solver`. If we determine `PoisFFT` requires more through inspection, we will incorporate the measurement of individual steps into `PoisFFT` as well.

Distributed Solvers Evaluation

We won't evaluate the `MPI + OpenMP` hybrid version of `PoisFFT` due to its instability. During our initial measurements the program crashed with several configurations. The reason could be incorrect implementation of `PoisFFT` for those configurations or even incorrect implementation of `PFFT` (the underlying FFT library used for `MPI` builds) whose hybrid `MPI + OpenMP` version is still under development.

For distributed solvers, we will measure the same metrics as for local solvers. We will measure a subset of the configurations. That is: strong scaling on 3D grids of side size 512 and weak scaling on 3D grids with local side size 256. Distributed solvers won't realistically be run on smaller problems and therefore doesn't make sense to measure them in those scenarios. We will also remove BC configurations that `PoisFFT` doesn't

implement for a distributed environment, namely: NsPP and PNsP, as well as BC configurations which crashed `PoisFFT` during our benchmarks, namely: NsDsNsNsNsNs and NDNNNN. Furthermore, we noticed that some configurations won't compute the correct result with Z axis being distributed in `PoisFFT` (but will compute the correct result for `poisson-solver` in the same environment), this includes DsDsDsDsNsDs, DsDsDsDsNsNs, DsDsDsDsPP and DDDDND. Those will be measured but with number of processes in the Z axis set to 1 for both `poisson-solver` and `PoisFFT`.

We also decrease the number of iterations per one configuration to 30 to reduce the overall measurement time (one iteration can take multiple seconds).

ELMM Evaluation

At last, we will evaluate the overall speedup of `ELMM` with the new solver. We will us the outputs of `ELMM`. It measures the overall execution time as well as time spent in the Poisson solver.

Benchmark Descriptions

The benchmark source code and runner scripts for `poisson-solver` can be found in the `benchmarks` branch of `poisson-solver`. The benchmark source code and runner scripts for `PoisFFT` can be found in my `PoisFFT` fork.

It is important to note that `PoisFFT` was modified for these benchmarks. During the initial analysis, we found that `PoisFFT` wasn't correctly linking and afterwards initializing `fftwf`. This resulted in slower execution time for `PoisFFT` in `float` precision configurations. The linking fix can be found here and the initialization fix here.

Local Solvers

The source code for `poisson-solver` is in `poisson-solver/tests/benchmarks/benchmark.cpp` and for `PoisFFT` in `PoisFFT/src/benchmark.f90`. The runner script for `poisson-solver` is in `poisson-solver/local_benchmark.sh` and for `PoisFFT` in `PoisFFT/local_benchmark.sh` (both have their `chimera` SLURM launchers in `run_local_benchmark.sh`).

The runner scripts loop through all the aforementioned combinations of configurations and for each one launch the benchmark. The benchmark then loops over all given grid sizes and for each one constructs the solver and repeatedly measures the aforementioned metrics of one iteration (for given amount of iterations).

poisson-solver

The total time is measured using `std::steady_clock` as this clock is guaranteed to be monotonic (unlike `std::high_resolution_clock` which may be aliased to non-monotonic `std::system_clock`). The buffer copy time and time to execute our own kernels is measured using `CUDAScopedTimer` and everything else using `HybridScopedTimer`. Both scoped timers accept a reference in their constructor into which they accumulate the measured time. Both also start the measurement in the constructor and stop it in the destructor.

`CUDAScopedTimer` uses CUDA events to measure the time of operations submitted into the CUDA stream (i.e., strictly GPU-related operations).

`HybridScopedTimer` uses `cudaDeviceSynchronize()` along with `std::steady_clock` to measure the time of operations executed by both CPU and GPU. By calling `cudaDeviceSynchronize()` before starting and then before stopping the measurement, we ensure we measure only the GPU kernels and all other CPU operations in the given scope. The `HybridScopedTimer` is utilized for measurements where we can't guarantee only CPU or only GPU usage and need to therefore measure both (e.g., calls to an external library).

PoisFFT

The total time is measured using `SYSTEM_CLOCK`.

Distributed Solvers

The source code for `poisson-solver` is in `poisson-solver/tests/benchmarks/benchmarks_mpi.cpp` and for `PoisFFT` in `PoisFFT/src/benchmarks_mpi.f90`. The runner scripts are `strong_scaling_benchmarks.sh` and `weak_scaling_benchmarks.sh` in their respective directories. The SLURM launchers are `run_strong_scaling_benchmarks.sh` and `run_weak_scaling_benchmarks.sh` in their respective directories.

The runners and SLURM launchers for distributed benchmarks are very similar to the local ones. The only significant difference is that in a distributed environment the iteration time is measured for every process. After each iteration we select the maximum total time among all processes and report that. In the case of `poisson-solver` the reported times of individual steps are acquired from this worst-case time.

The rationale behind being that after the solver is ran, there will most likely be some synchronization point (e.g. `MPI_Barrier`) before the application continues. This approach allows us to measure the time until all processes finish without measuring the overhead of synchronization.

The DsDsDsDsNsDs, DsDsDsDsNsNs, DsDsDsDsPP and DDDDND configurations were measured manually with Z axis non-distributed (i.e. only Y distributed).

Platform

The local solver benchmarks were compiled and executed on the `bw01` node on `chimera` cluster. `bw01` has: 2x NVIDIA RTX PRO Blackwell Server Edition, PCIe 96 GB (only one is used for local benchmarks). The CPU on `bw01` is an AMD EPYC 9454 48-Core Processor with 768 GB RAM and supporting avx, avx2, avx512 vector ISA. The CUDA version on `bw01` is 13.3.

The distributed solver benchmarks were compiled on `volta05` and executed on nodes `volta02`, `volta03` and `volta05`. For their combination, see the SLURM launcher scripts (`run_...`).

The `chimera` cluster has only one Blackwell node and therefore `volta` nodes were used exclusively to prevent measurements with significant load imbalance among nodes.

`volta02` and `volta03` both have: 2x NVIDIA V100 PCIe 16 GB GPU, Intel Xeon Silver 4110 CPU with 2 sockets of 8 cores each and 192 GB RAM. `volta05` has: 4x NVIDIA V100 SXM2 32 GB, Intel Xeon Gold 5218 CPU with 2 sockets of 16 cores each and 384 GB RAM. All `volta[02,03,05]` CPUs support avx, avx2 and avx512. The CUDA version on all `volta[02,03,05]` is 12.9.

The `volta` nodes are connected with 10Gb Ethernet (no InfiniBand).

Analysis

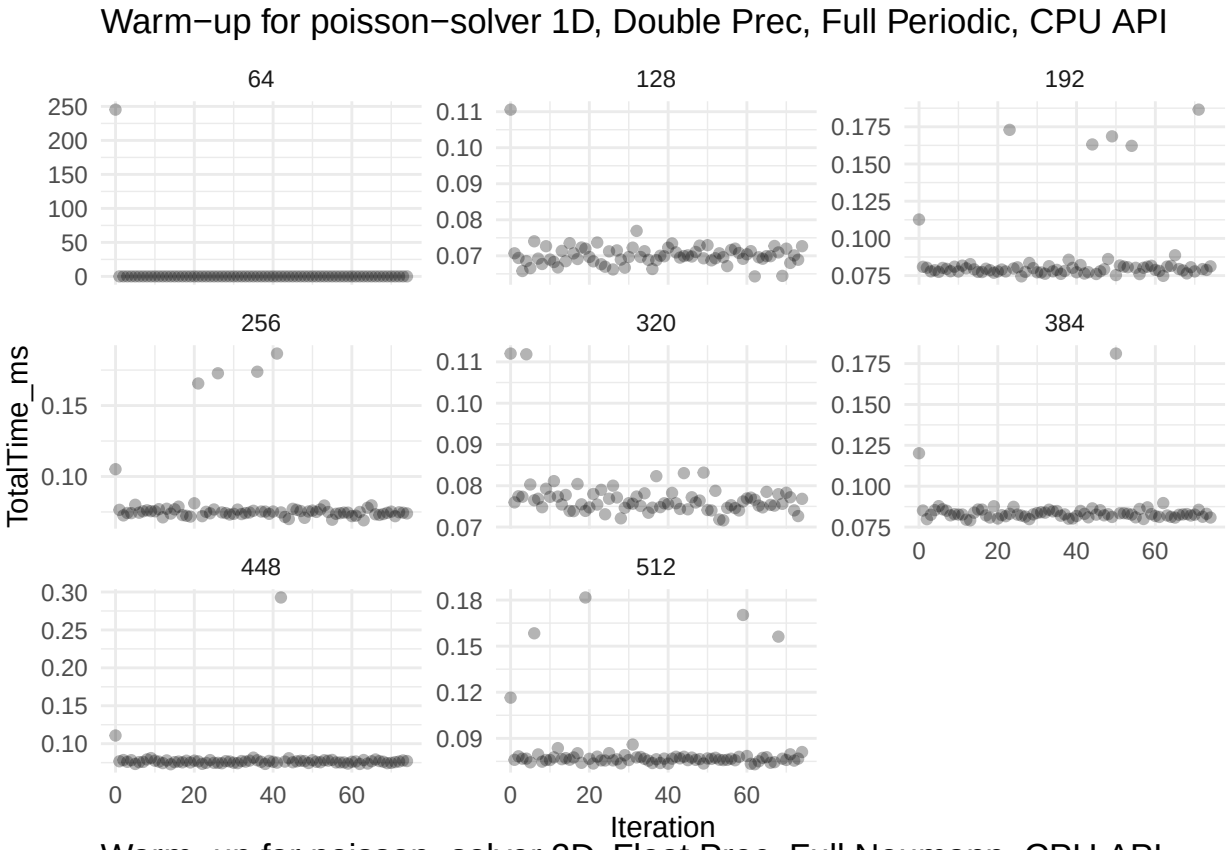
Local Solvers

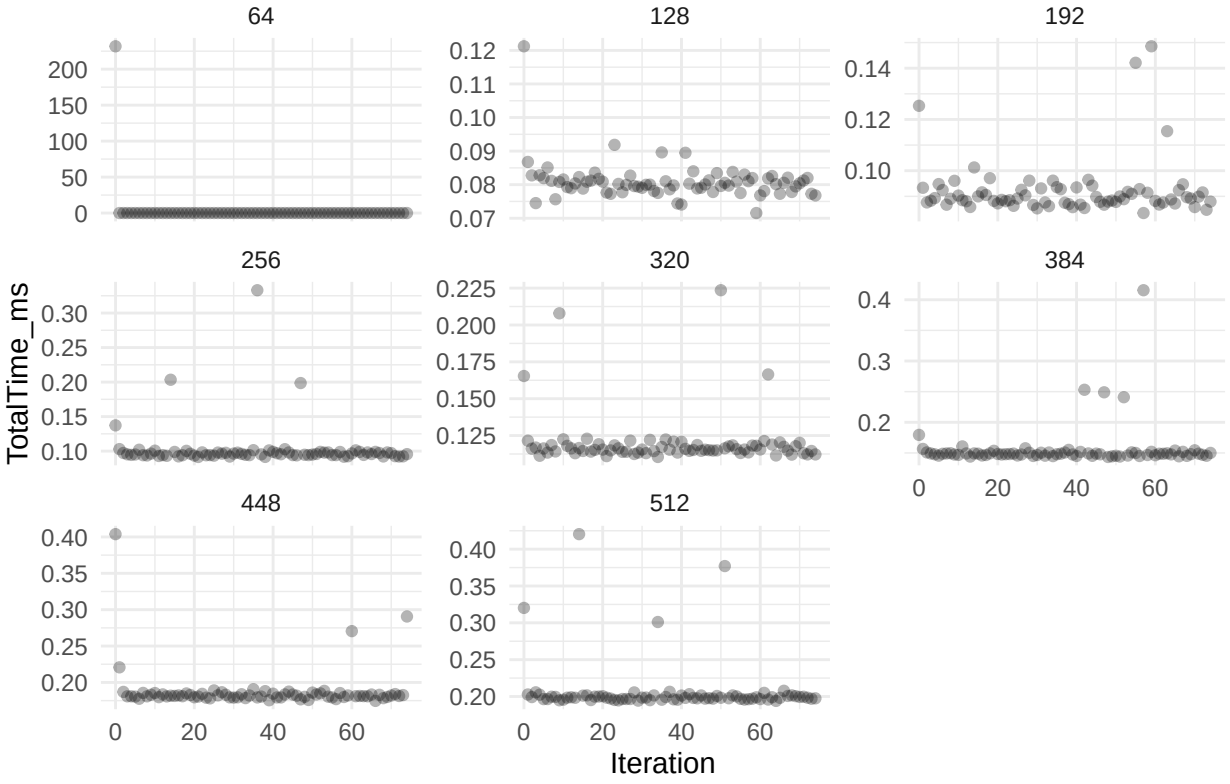
Warm-up and Outliers

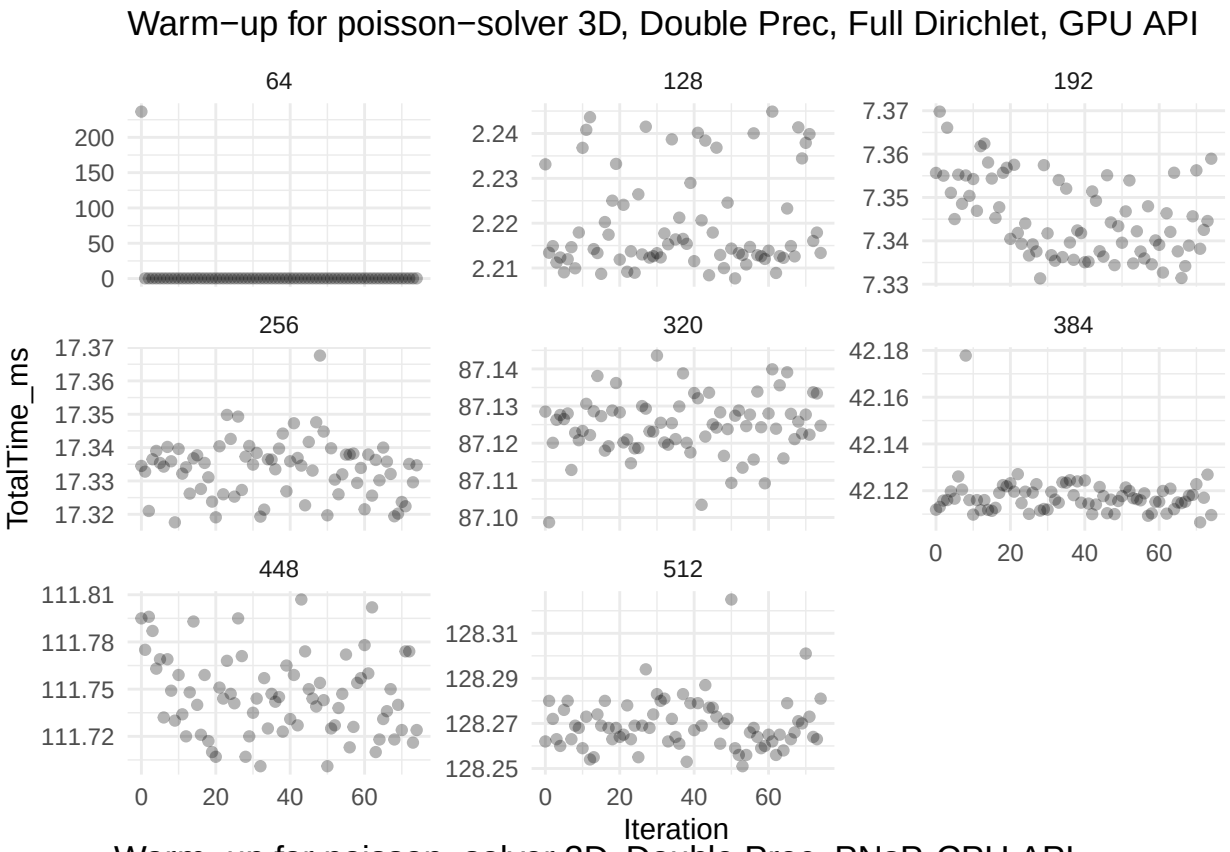
First, we will manually determine the number of warm-up iterations for pre-selected configurations and discard that number of iterations from all configurations. The number of warm-up iterations should not differ significantly between different solver configurations.

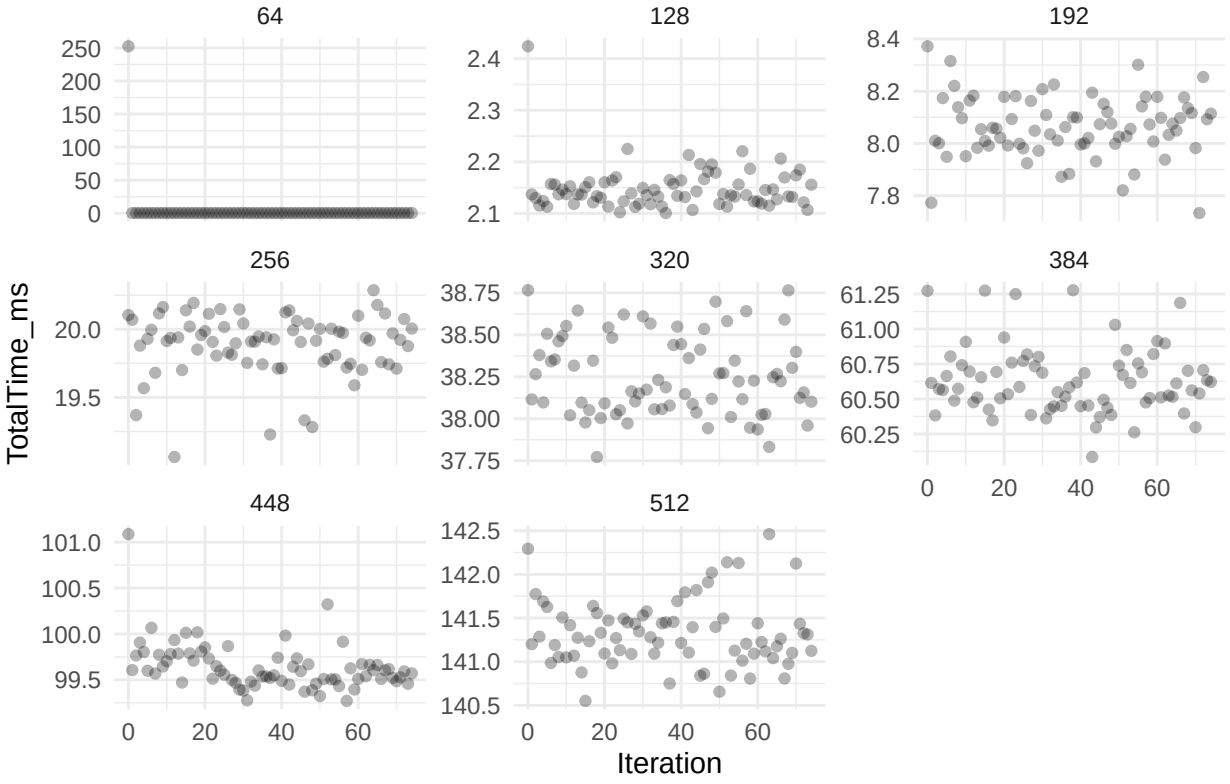
Afterwards, we will determine if there are any outliers and discord those as well.

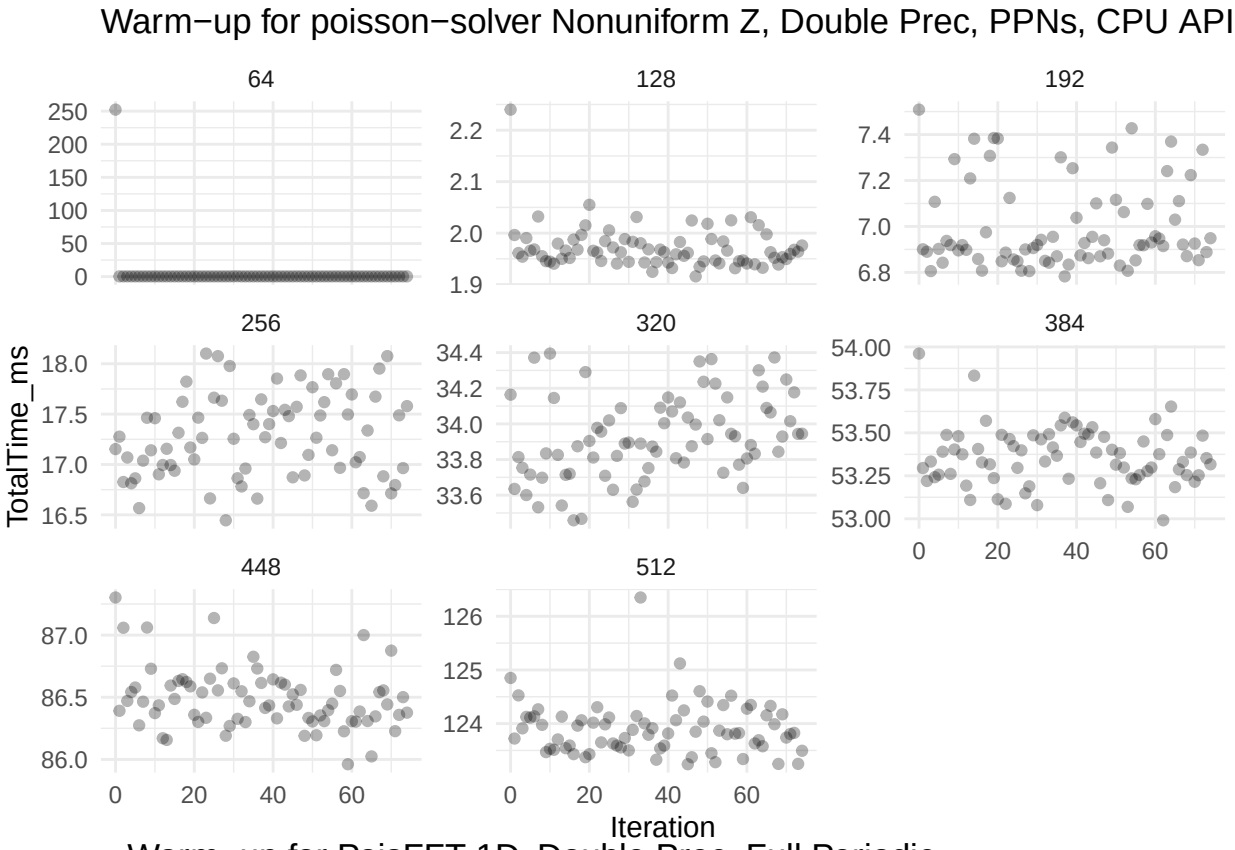
We expect there to be a one or two iteration warm-up. There may be more if FFT libraries perform autotuning after the initial planning phase.

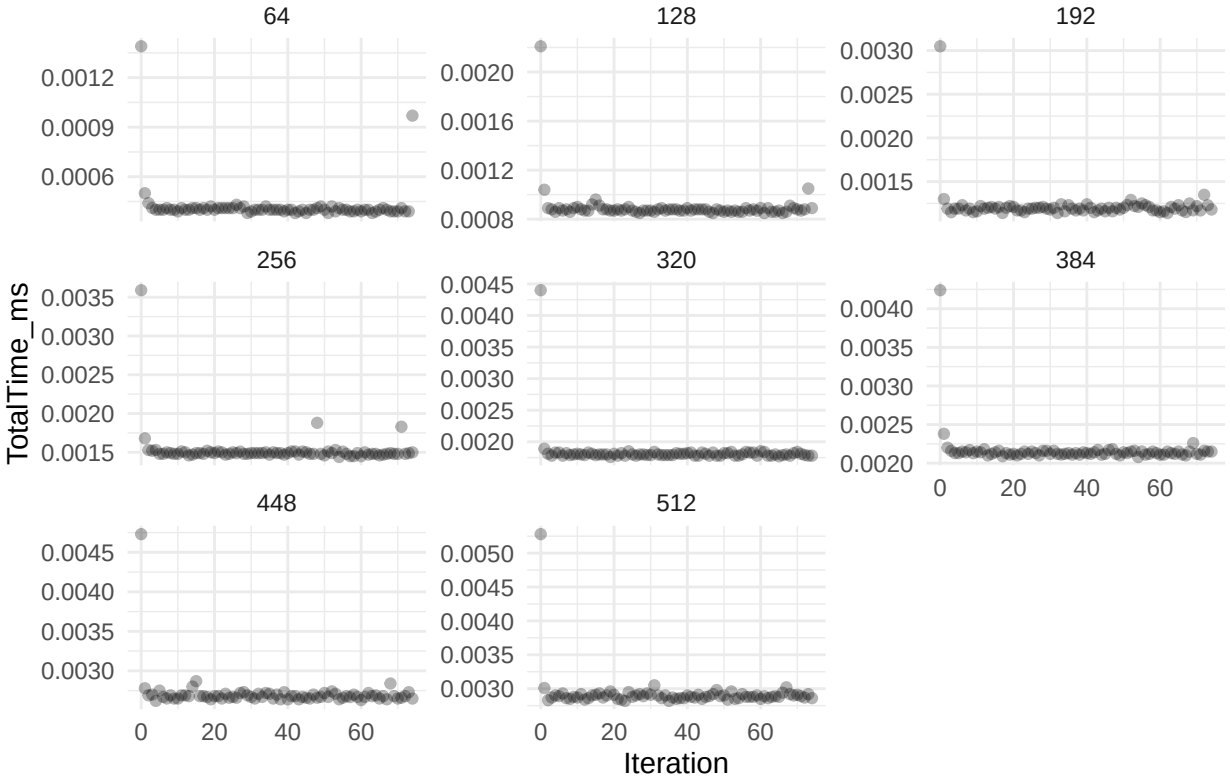

Warm−up for poisson−solver 1D, Double Prec, Full Periodic, CPU API
64
128
192
256
320
384
448
512
TotalTime_ms
Iteration

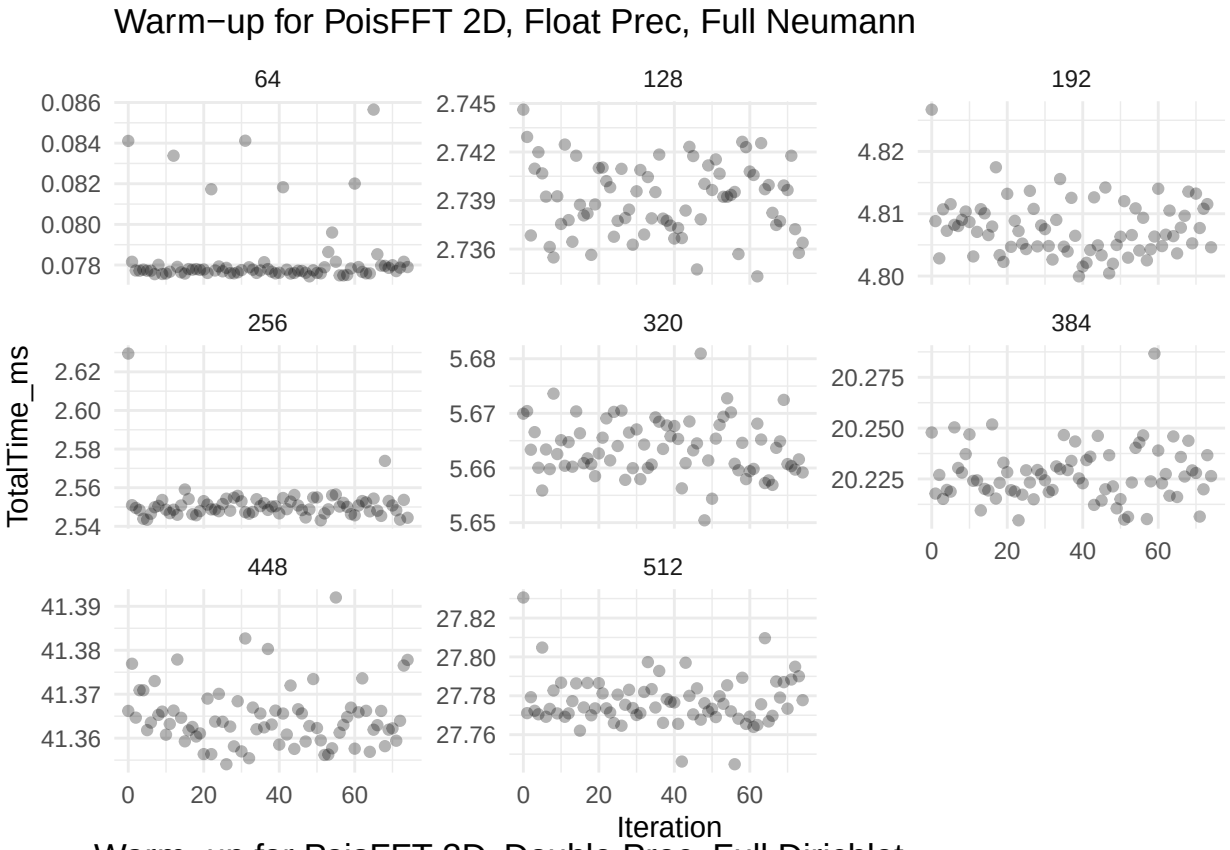

Warm−up for poisson−solver 2D, Float Prec, Full Neumann, CPU API
64
128
192
256
320
384
448
512
TotalTime_ms
Iteration

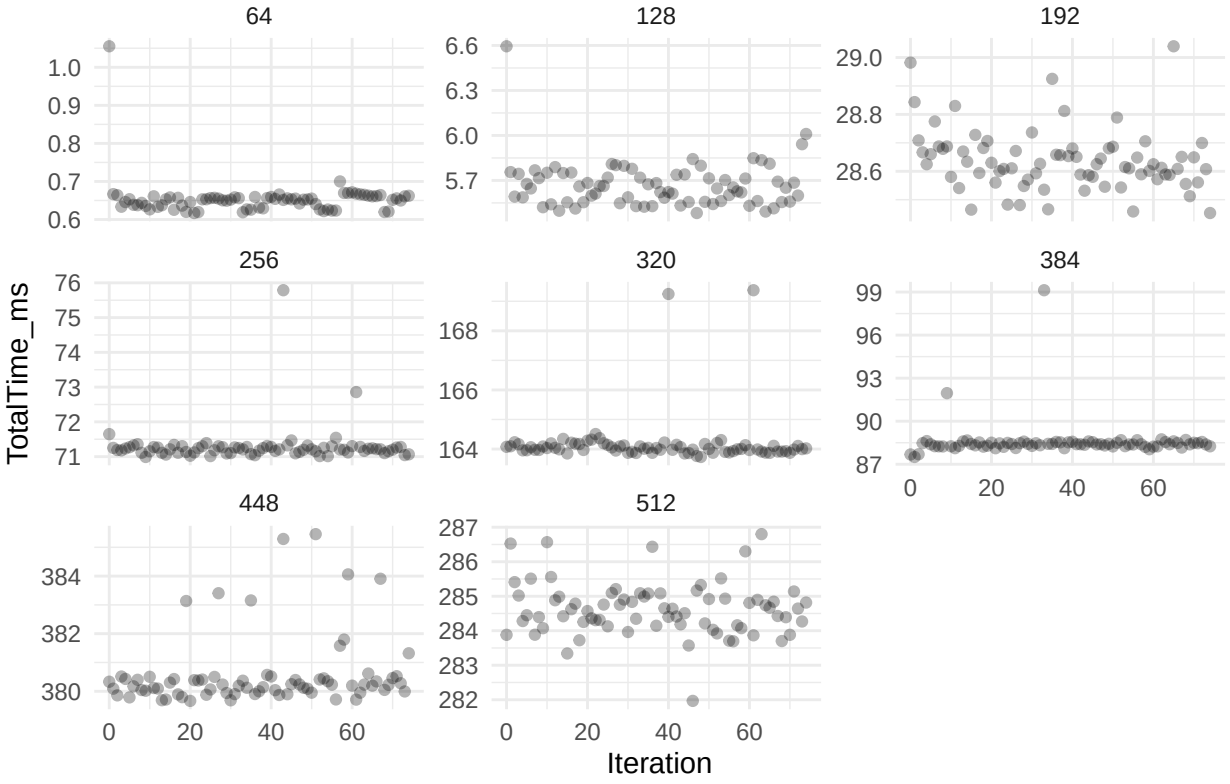

Warm−up for poisson−solver 3D, Double Prec, Full Dirichlet, GPU API
64
128
192
256
320
384
448
512
TotalTime_ms
Iteration

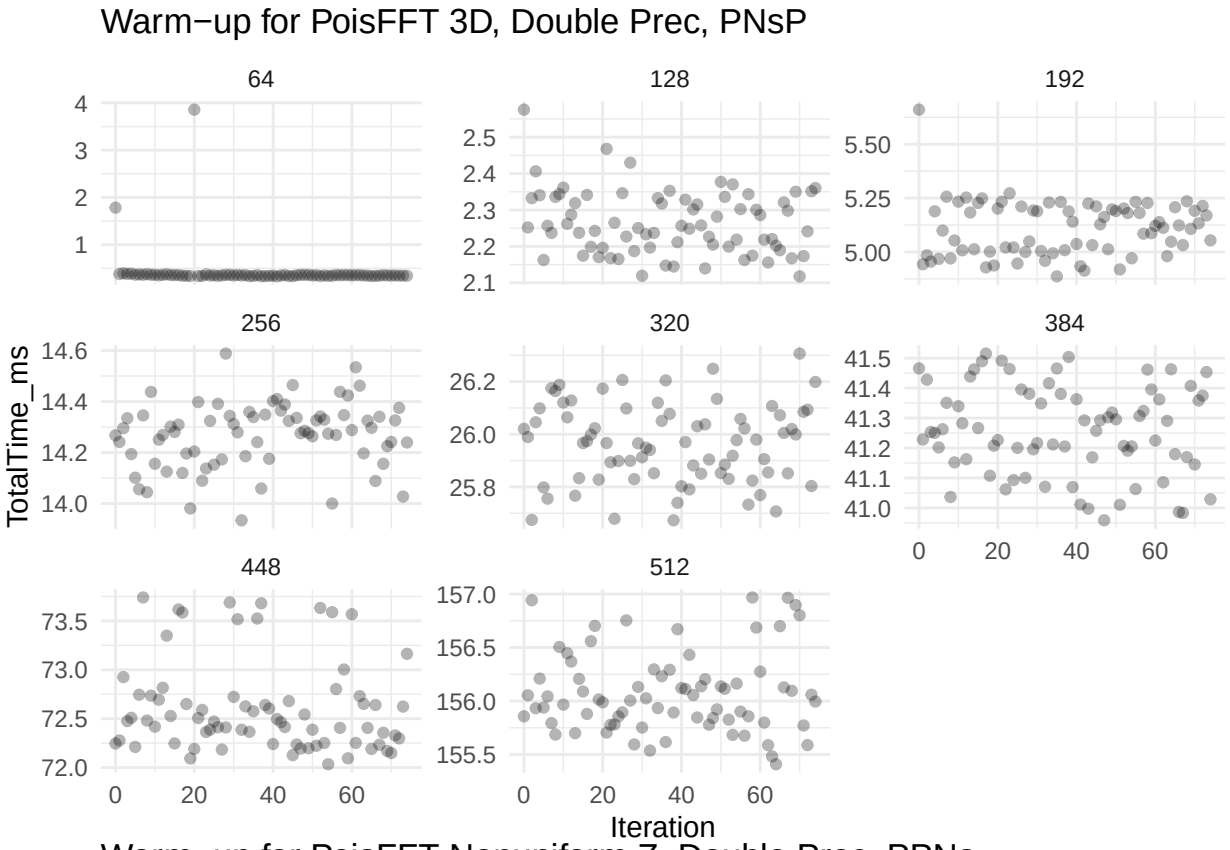

Warm−up for poisson−solver 3D, Double Prec, PNsP, CPU API
64
128
192
256
320
384
448
512
TotalTime_ms
Iteration

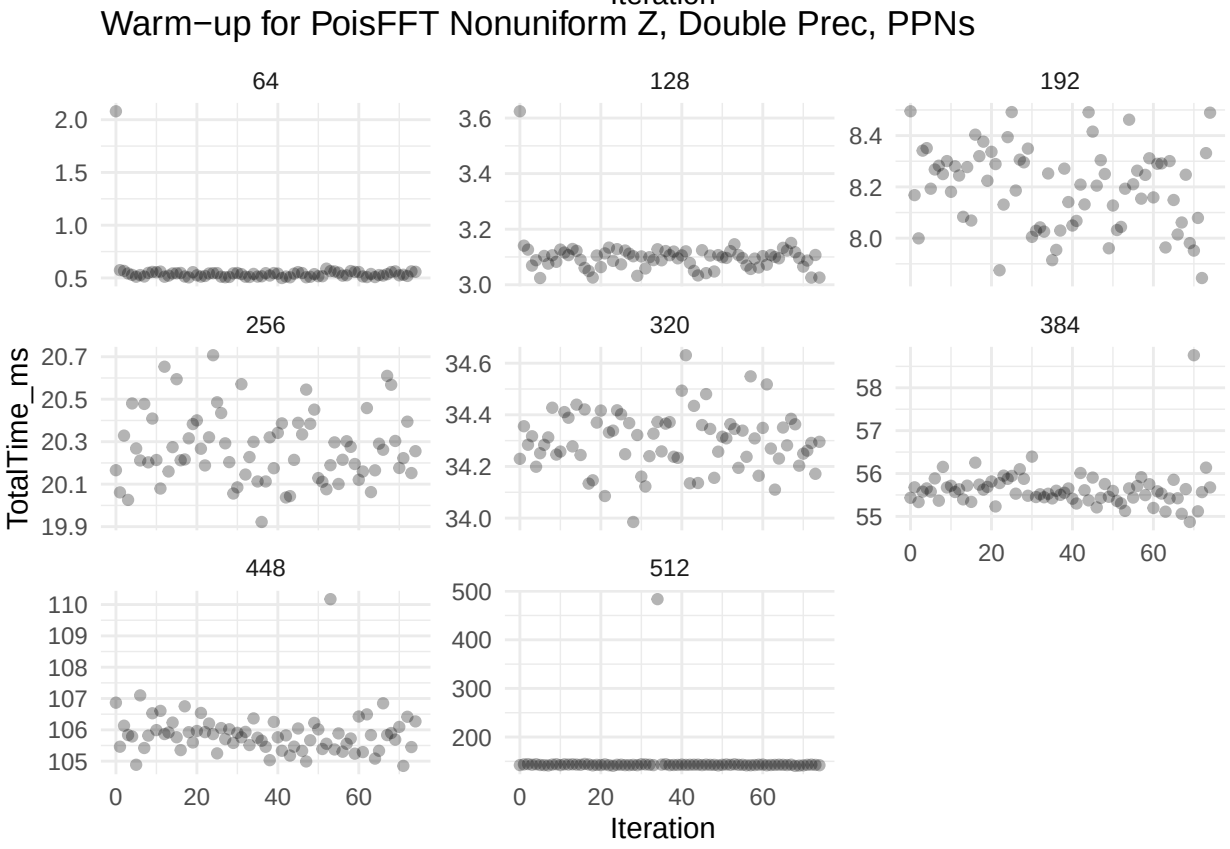

Warm−up for poisson−solver Nonuniform Z, Double Prec, PPNs, CPU API
64
128
192
256
320
384
448
512
TotalTime_ms
Iteration


Warm−up for PoisFFT 1D, Double Prec, Full Periodic
64
128
192
256
320
384
448
512
TotalTime_ms
Iteration


Warm−up for PoisFFT 2D, Float Prec, Full Neumann
64
128
192
256
320
384
448
512
TotalTime_ms
Iteration


Warm−up for PoisFFT 3D, Double Prec, Full Dirichlet
64
128
192
256
320
384
448
512
TotalTime_ms
Iteration





From the plots above it appears that the measurements have at most 2 warm-up iterations. We discard those

along with times greater than 3σ.

Finding Best `PoisFFT` Setup

For every `PoisFFT` configuration, we will select the number of cores that has the highest throughput average over all grid sizes.

Comparison to `poisson-solver`

We will average the times for each grid size and compare `poisson-solver` against `PoisFFT`. The results are saved into the `plots/local` directory.

For all 1D grids, `PoisFFT` outperforms `poisson-solver`. For grids of this size, GPUs rarely provide any benefit over CPUs and this outcome is expected.

For 2D grids, the trend is that on grid side sizes of 64 `PoisFFT` outperforms `poisson-solver` (even the GPU API) but on larger grids, `poisson-solver` is faster. Here, we can also start to observe the bottleneck of our computation - buffer copy times.

For 3D grids, `poisson-solver` outperforms `PoisFFT` at all times when GPU API is used. For most boundary condition configurations, `poisson-solver` performs better than `PoisFFT`. However, `PoisFFT` achieves better performance than the CPU API of `poisson-solver` even for large grids for some boundary condition configurations. This happens when the computation itself is trivial enough that the overhead of copying the data to and from the device are not mitigated by the speedup achieved by the GPU port.

We can also observe that sometimes, FFT computation takes longer than buffer copying (e.g. for 3D full dirichlet configuration).

Another interesting phenomenon is that sometimes it takes less time to compute a larger grid. The reason could be that the speed of FFT computation depends on the prime factorization of the grid size and therefore could result in faster algorithm being used on a larger grid leading to faster solve time than on a smaller grid.

The table in `tables/local_speedup_table.html` lists speedups for the plotted configurations.

The speedups on 3D grids of `poisson-solver` CPU API compared to `PoisFFT` are up to `~20x`. For GPU API the speedups reach `~114x`.

For 2D grids the speedups are more impressive, reaching up to `~227x` for the CPU API and `~645x` for the GPU API.

Conclusion for Local Solvers

While in many configurations `poisson-solver` (CPU API) provides significant speedup over `PoisFFT`, there still remain cases where the performance is identical or even worse. With manual examination, we confirmed that the buffer copy times between host and device are the main bottleneck, although FFT times are not to be ignored for certain configurations.

There is currently no other GPU FFT library that satisfies `poisson-solver` requirements. It may be worthwhile contributing to VkFFT to ensure maximum performance due to it currently being the only library to provide the same functionality as `FFTW` does on CPUs.

The copy bottleneck can be solved by converting more parts of the whole system to execute on the GPU and thus prevent the copies.

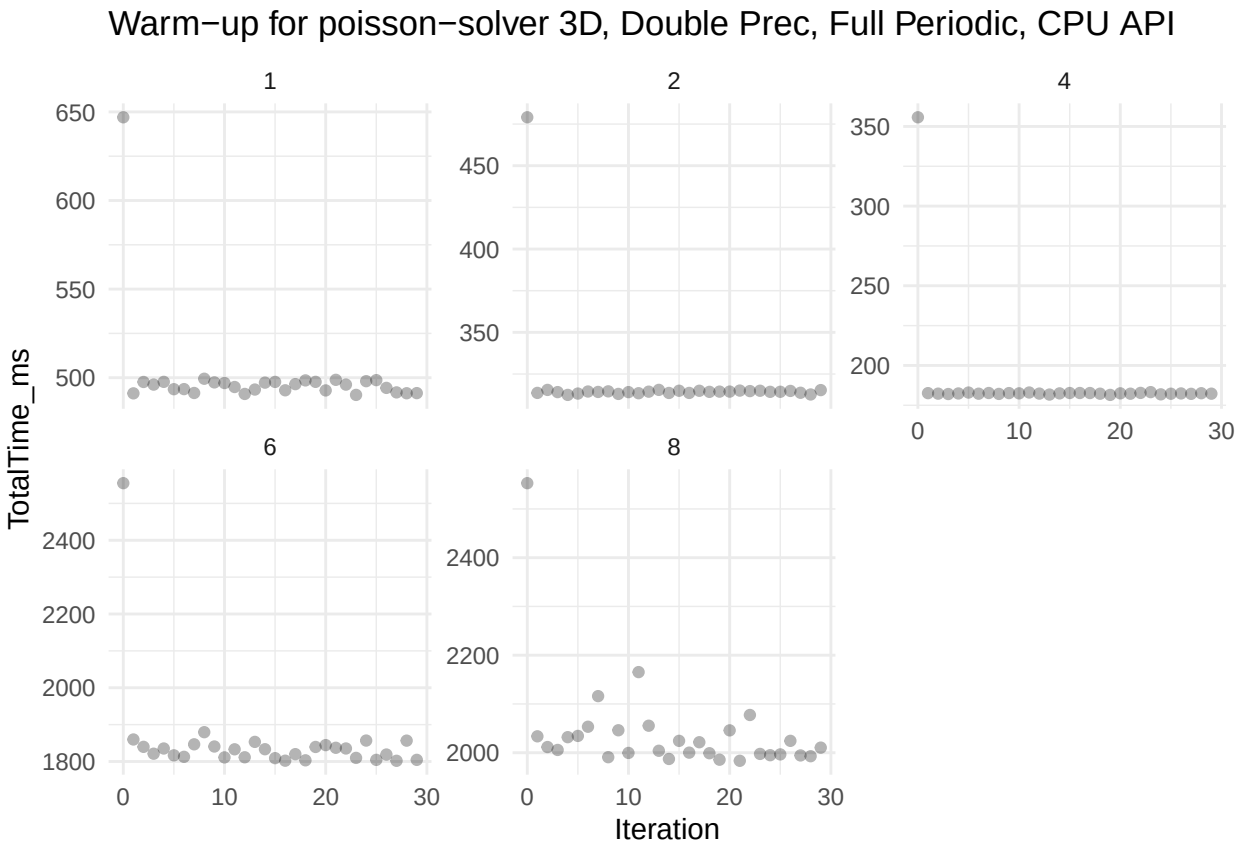
Converting legacy Fortran code bases manually is not optimal as their code readability can be rather poor (with little to no documentation) and manual conversions thus become very time-consuming. The preferred solution would be to design a system for transforming selected parts of a Fortran code base automatically into CUDA, a solution which is currently being worked on.

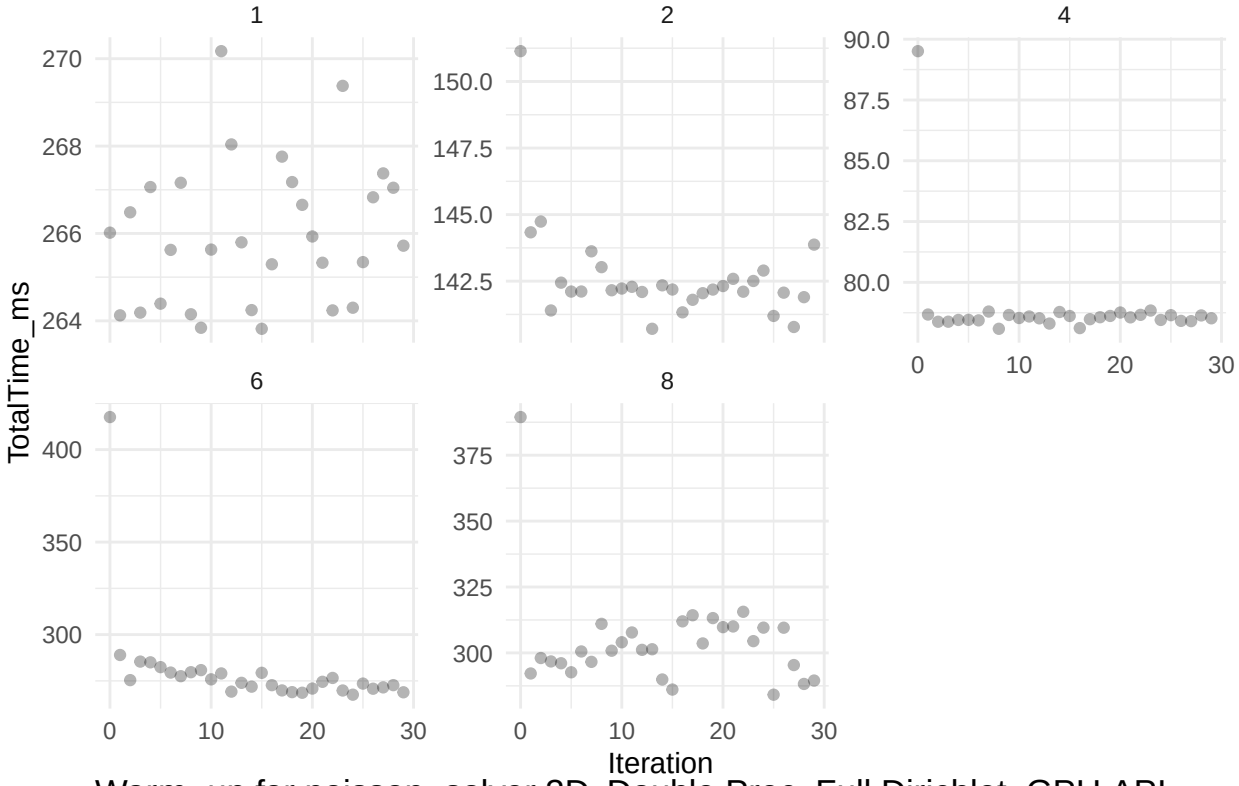
Distributed Solvers

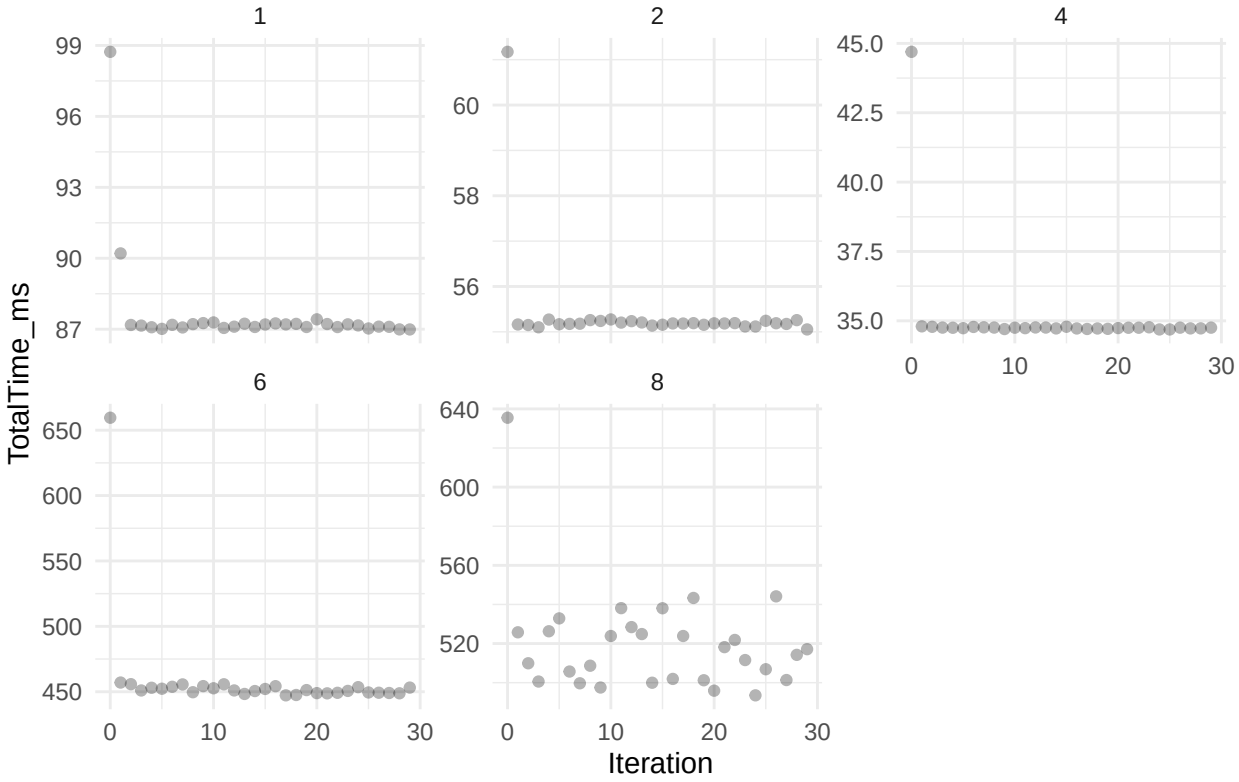
Warm-up and Outliers

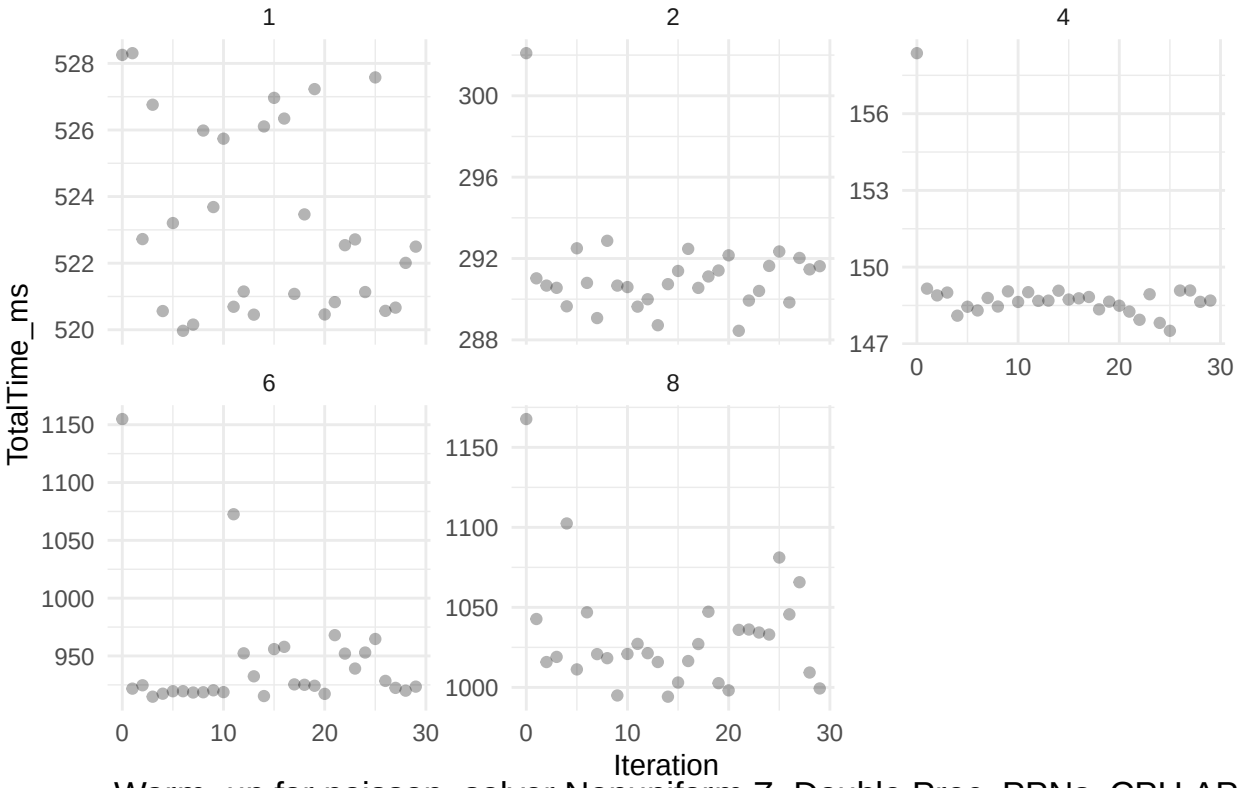
We will once again manually determine the number of warm-up iterations for pre-selected configurations, discard that number of iterations from all configurations and afterwards, discard any outliers.

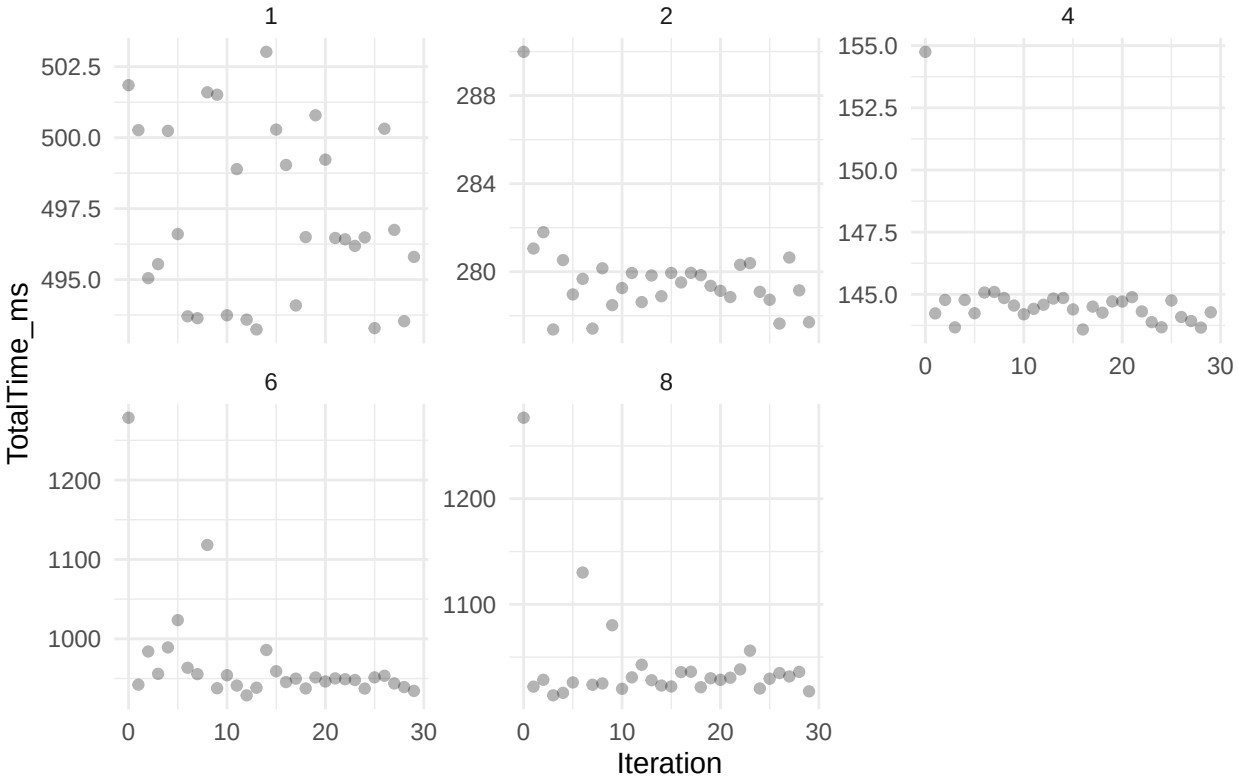
First, the strong scaling measurements.

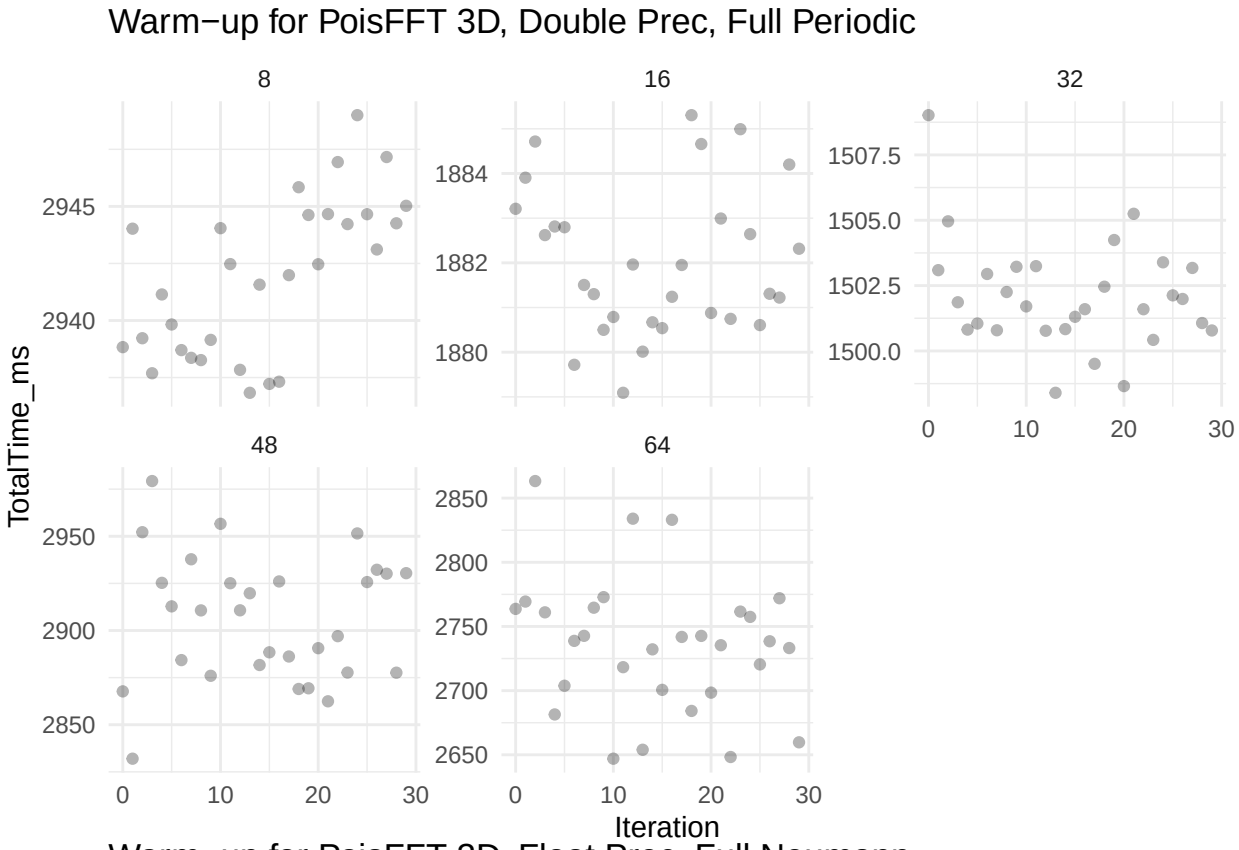


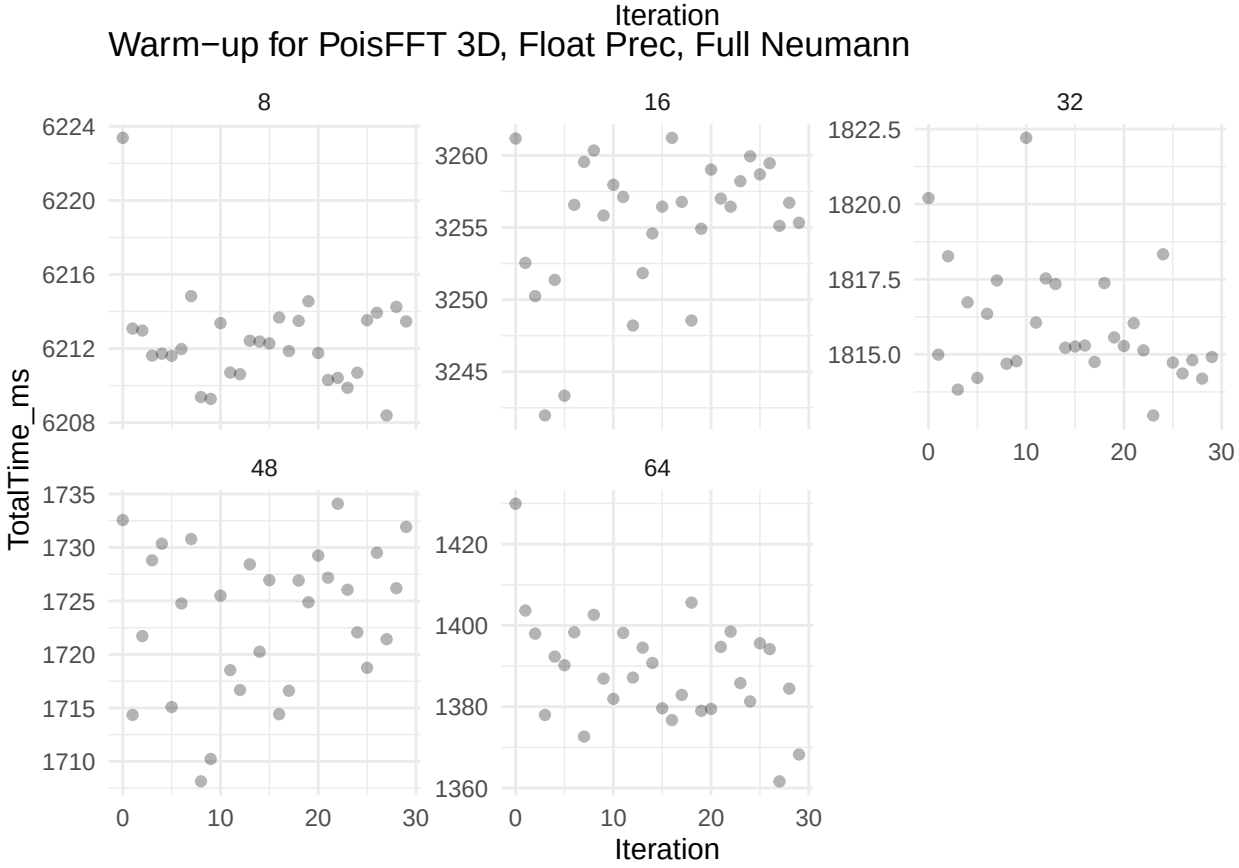

Warm−up for poisson−solver 3D, Float Prec, Full Neumann, CPU API
1
2
4
6
8
TotalTime_ms
Iteration

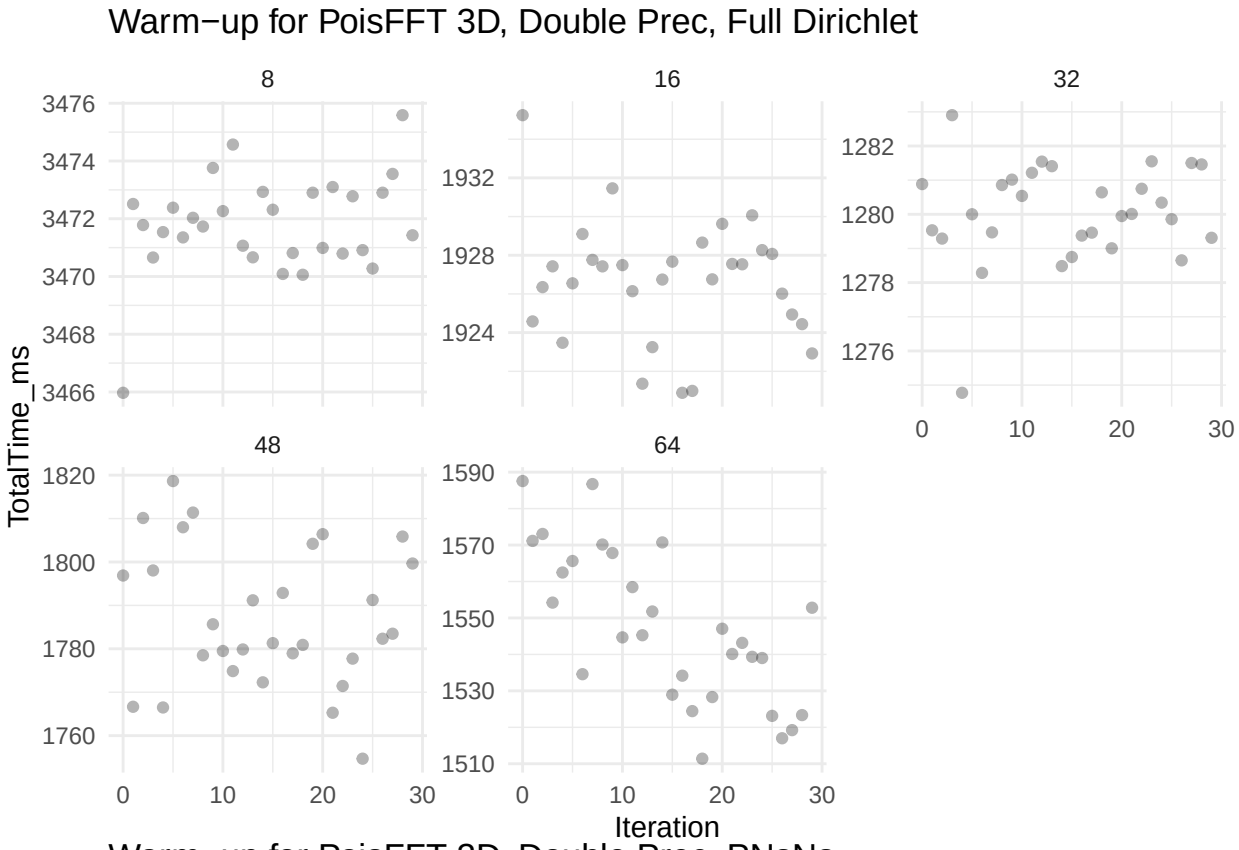

Warm−up for poisson−solver 3D, Double Prec, Full Dirichlet, GPU API
1
2
4
6
8
TotalTime_ms
Iteration

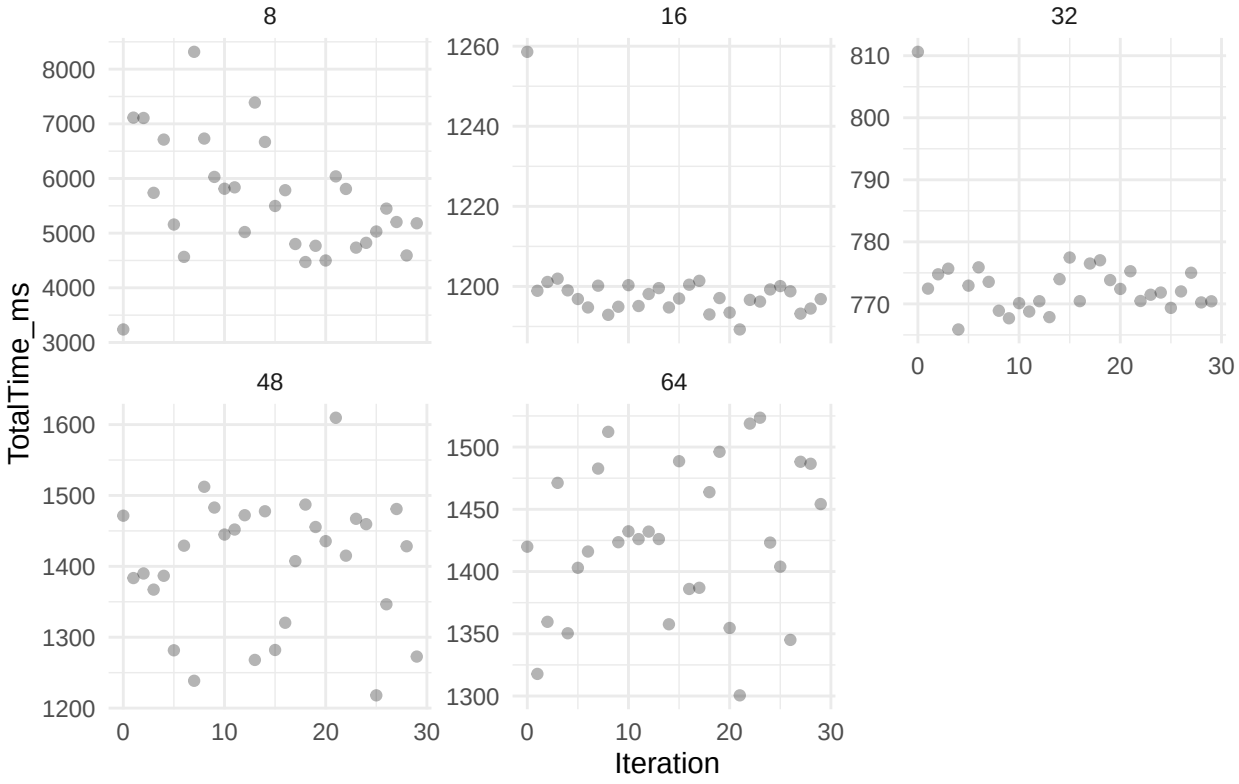

Warm−up for poisson−solver 3D, Double Prec, PNsNs, CPU API
1
2
4
6
8
TotalTime_ms
Iteration

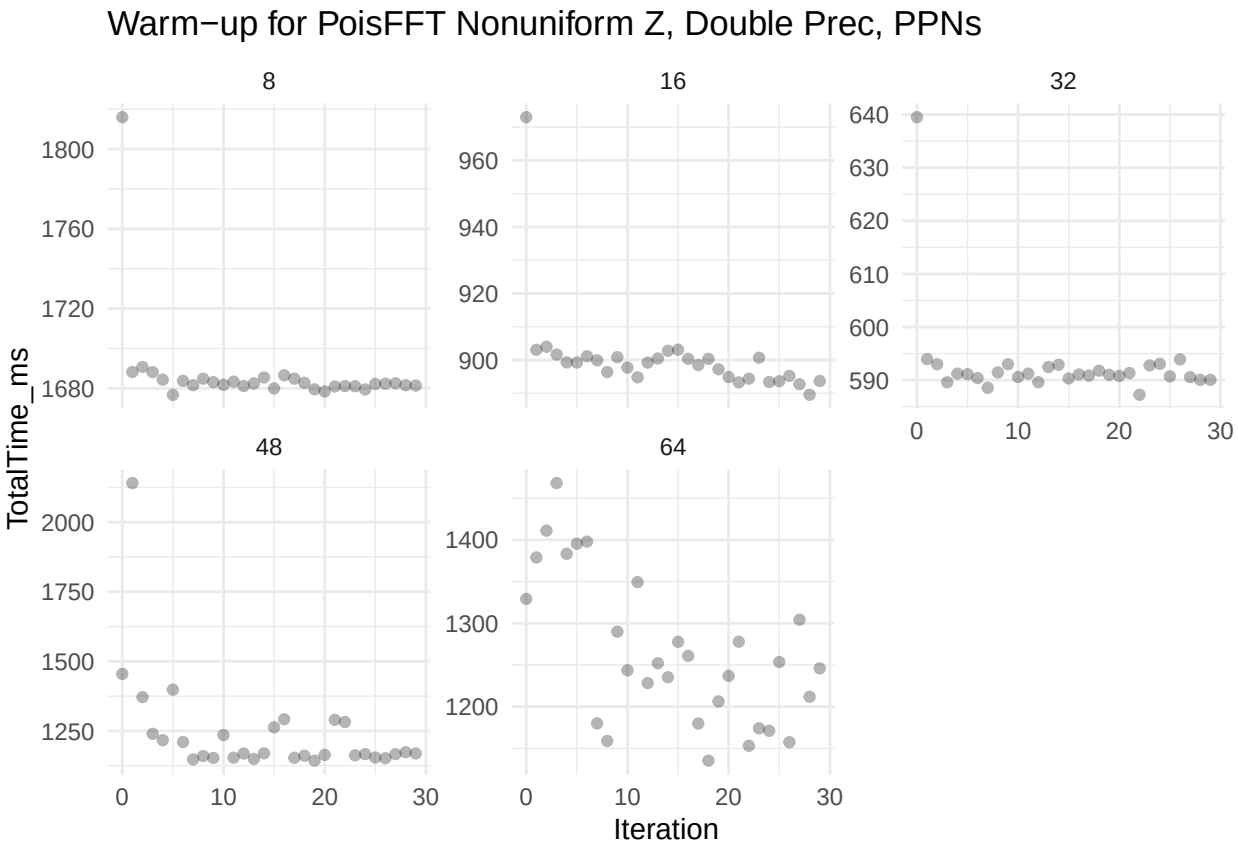

Warm−up for poisson−solver Nonuniform Z, Double Prec, PPNs, CPU AP
1
2
4
6
8
TotalTime_ms
Iteration


Warm−up for PoisFFT 3D, Double Prec, Full Periodic
8
16
32
48
64
TotalTime_ms
Iteration


Warm−up for PoisFFT 3D, Float Prec, Full Neumann
8
16
32
48
64
TotalTime_ms
Iteration

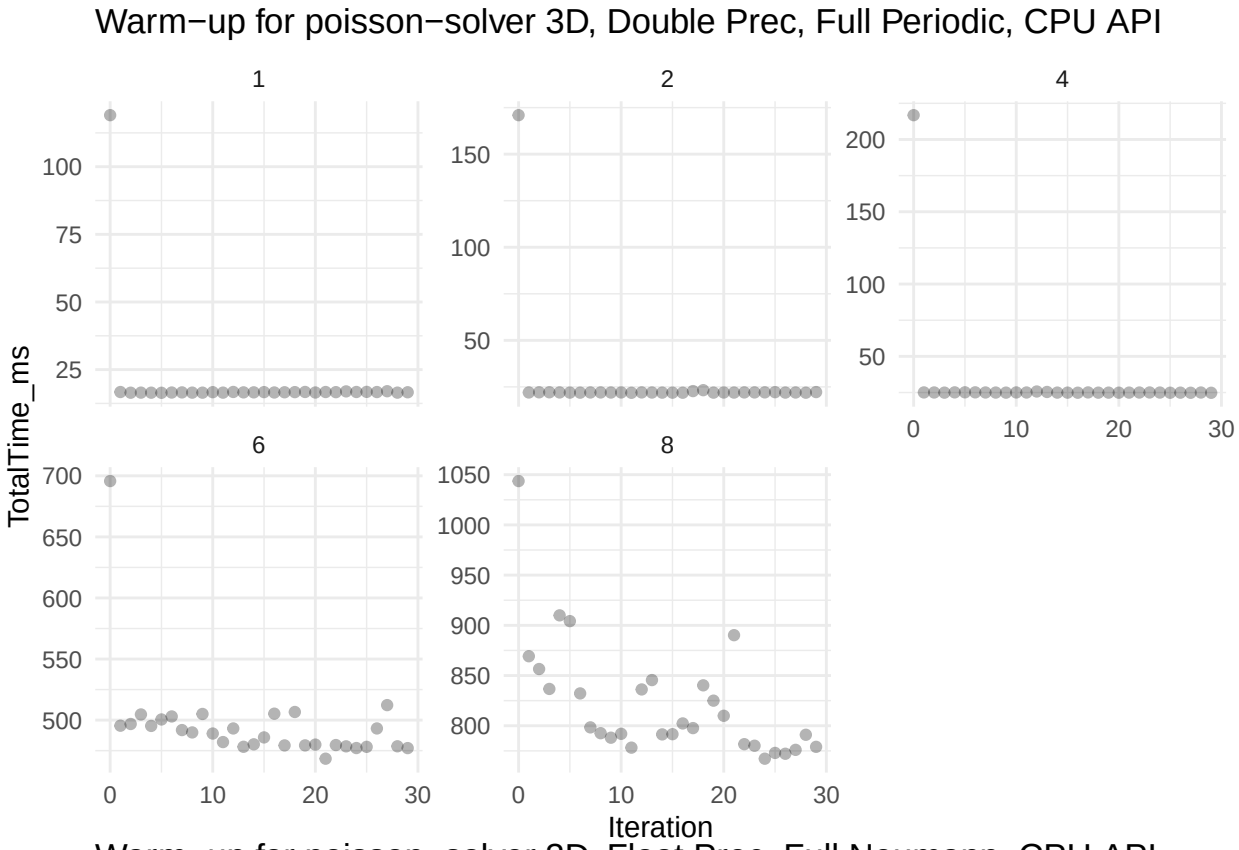

Warm−up for PoisFFT 3D, Double Prec, Full Dirichlet
8
16
32
48
64
TotalTime_ms
Iteration

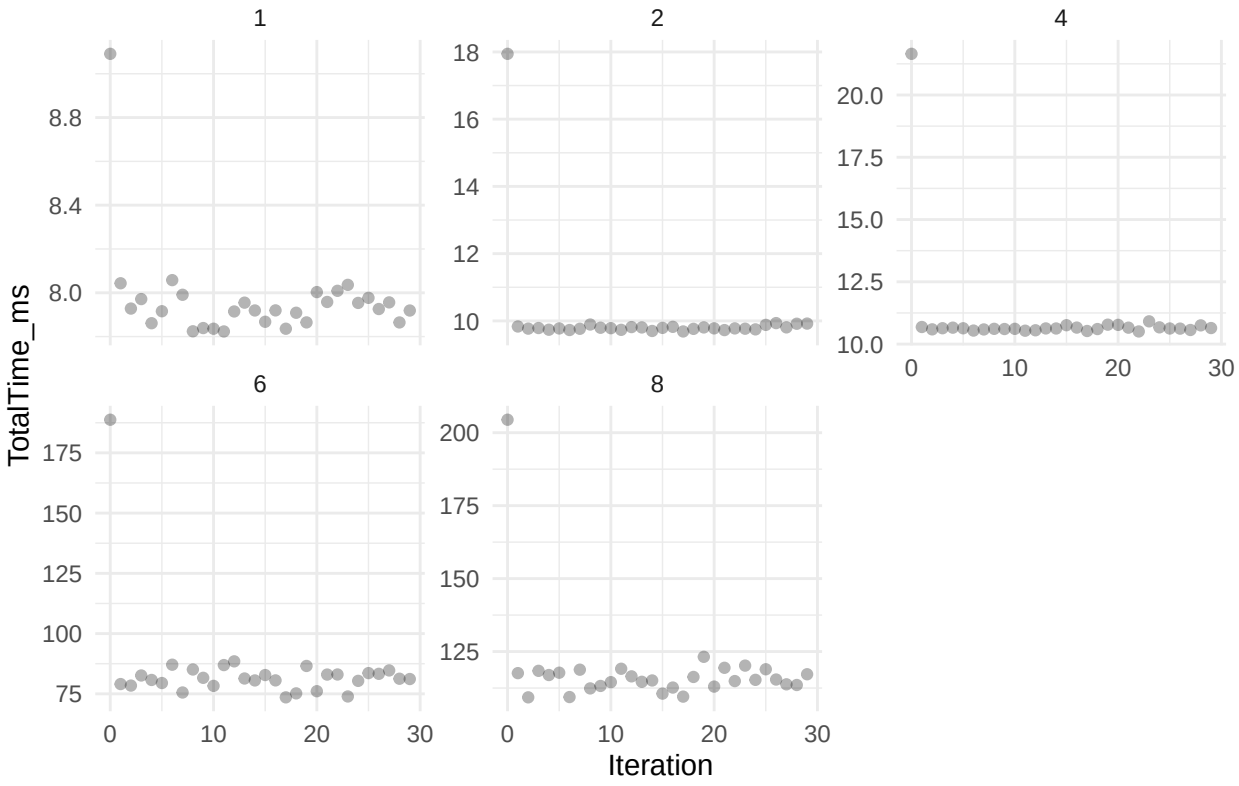

Warm−up for PoisFFT 3D, Double Prec, PNsNs
8
16
32
48
64
TotalTime_ms
Iteration

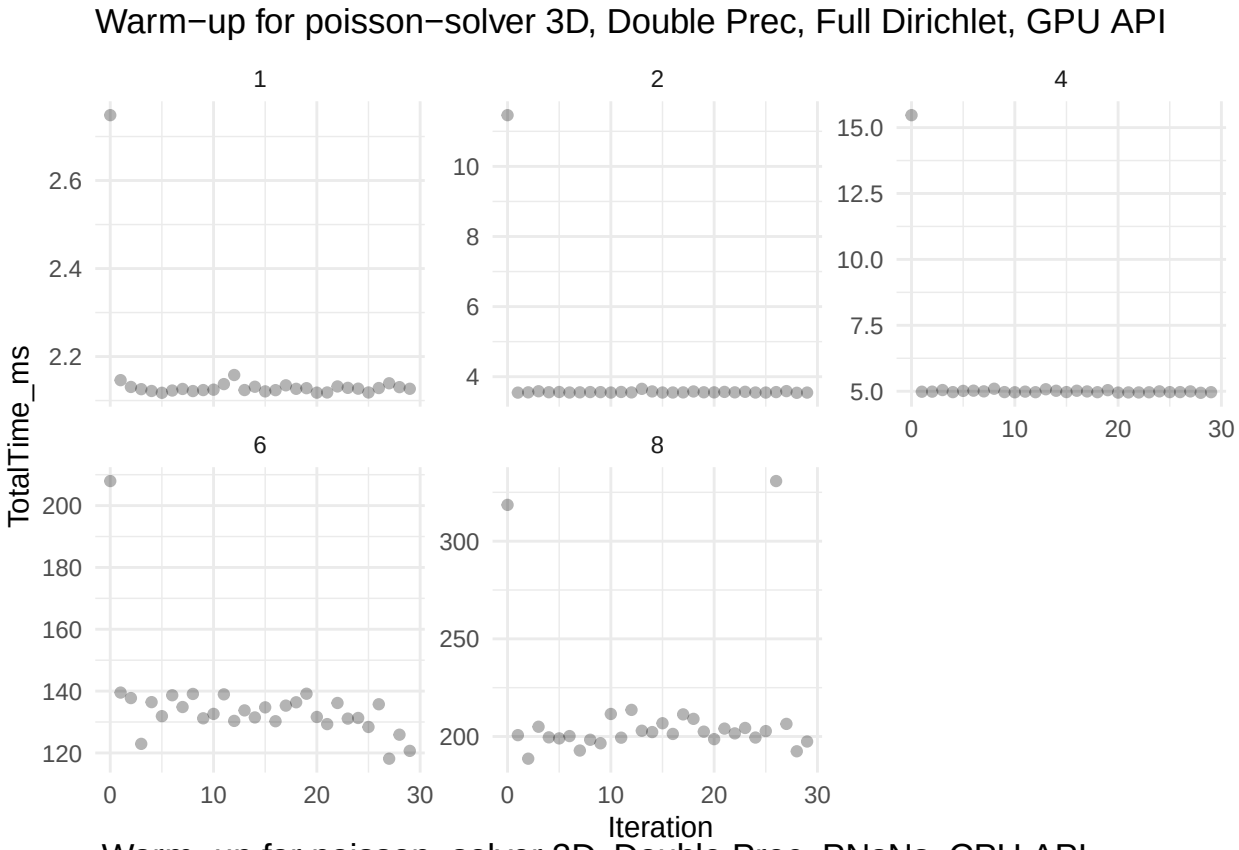


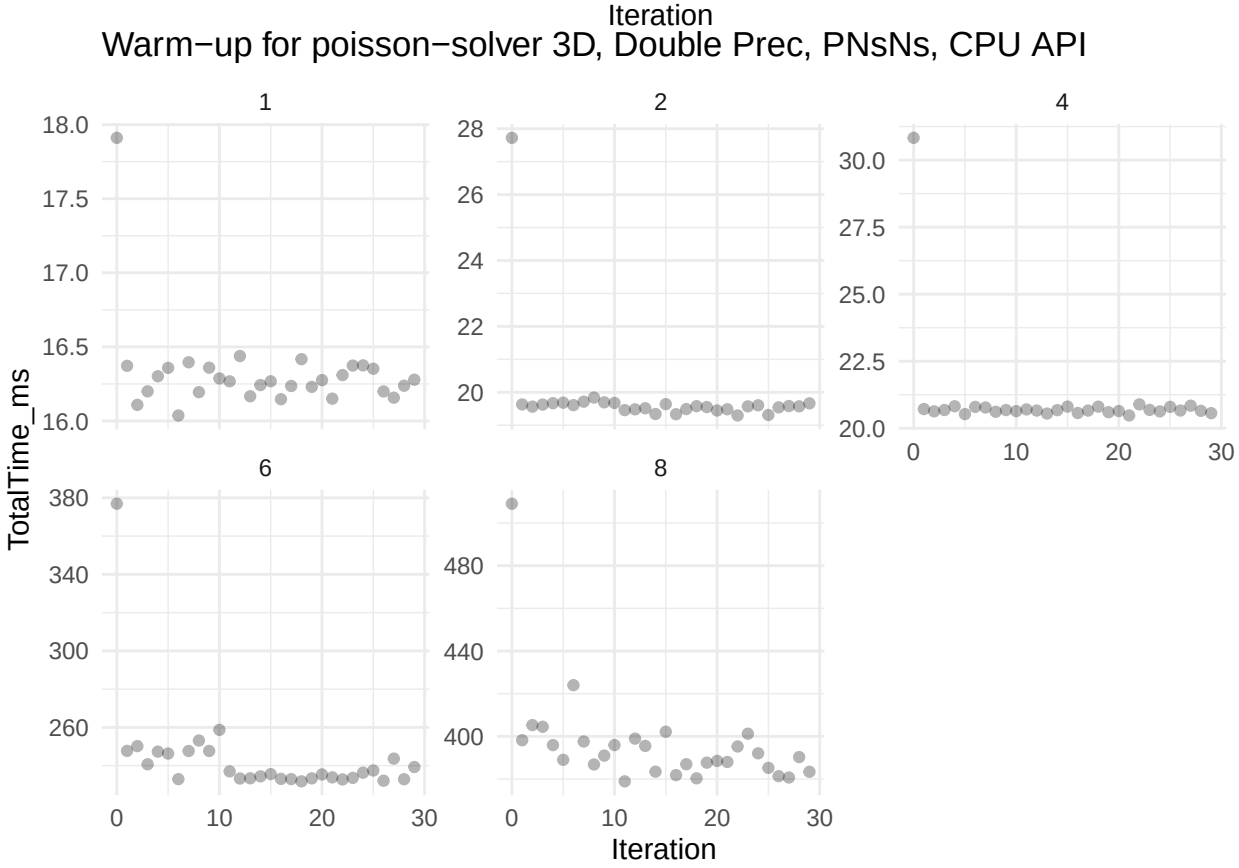
Here, it appears that the measurements have at most 4 warm-up iterations. We discard those along with times greater than 3σ.

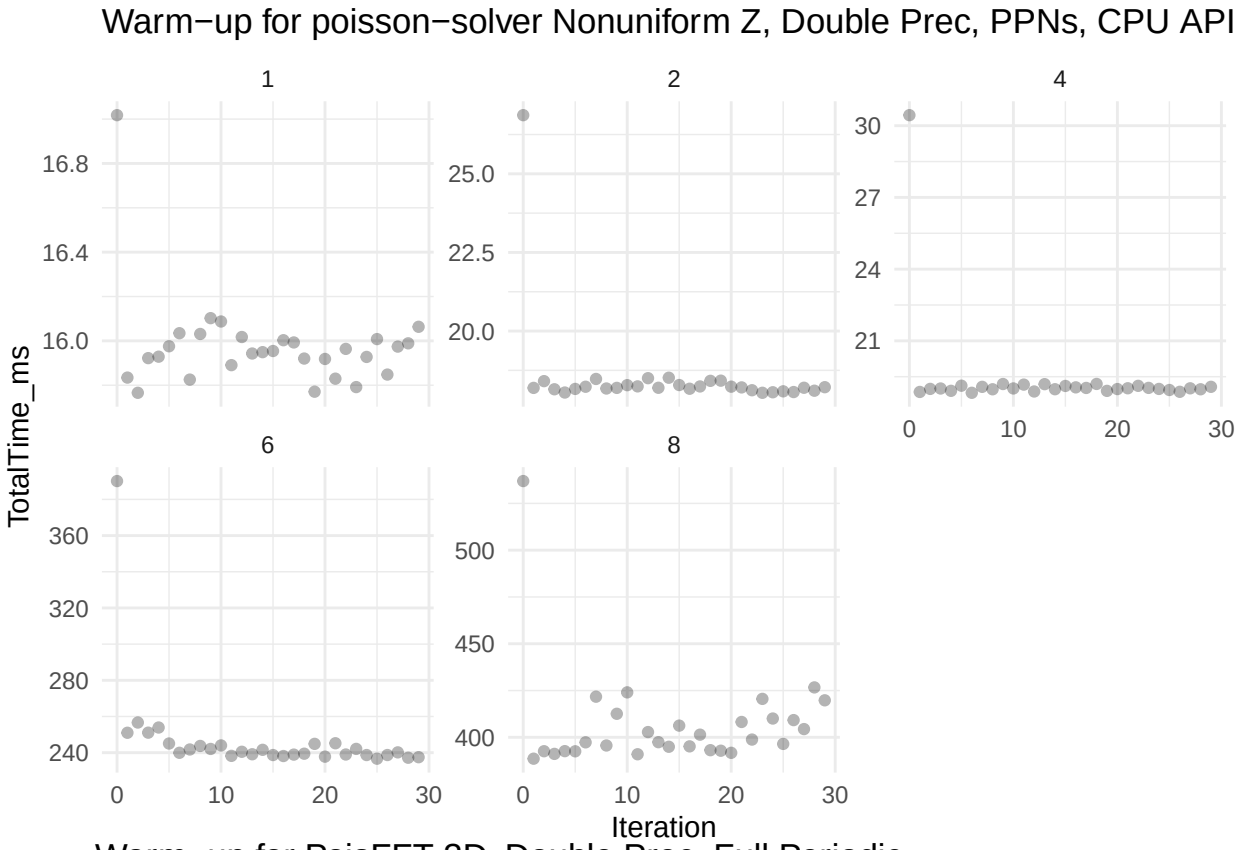
Next, the weak scaling measurements.

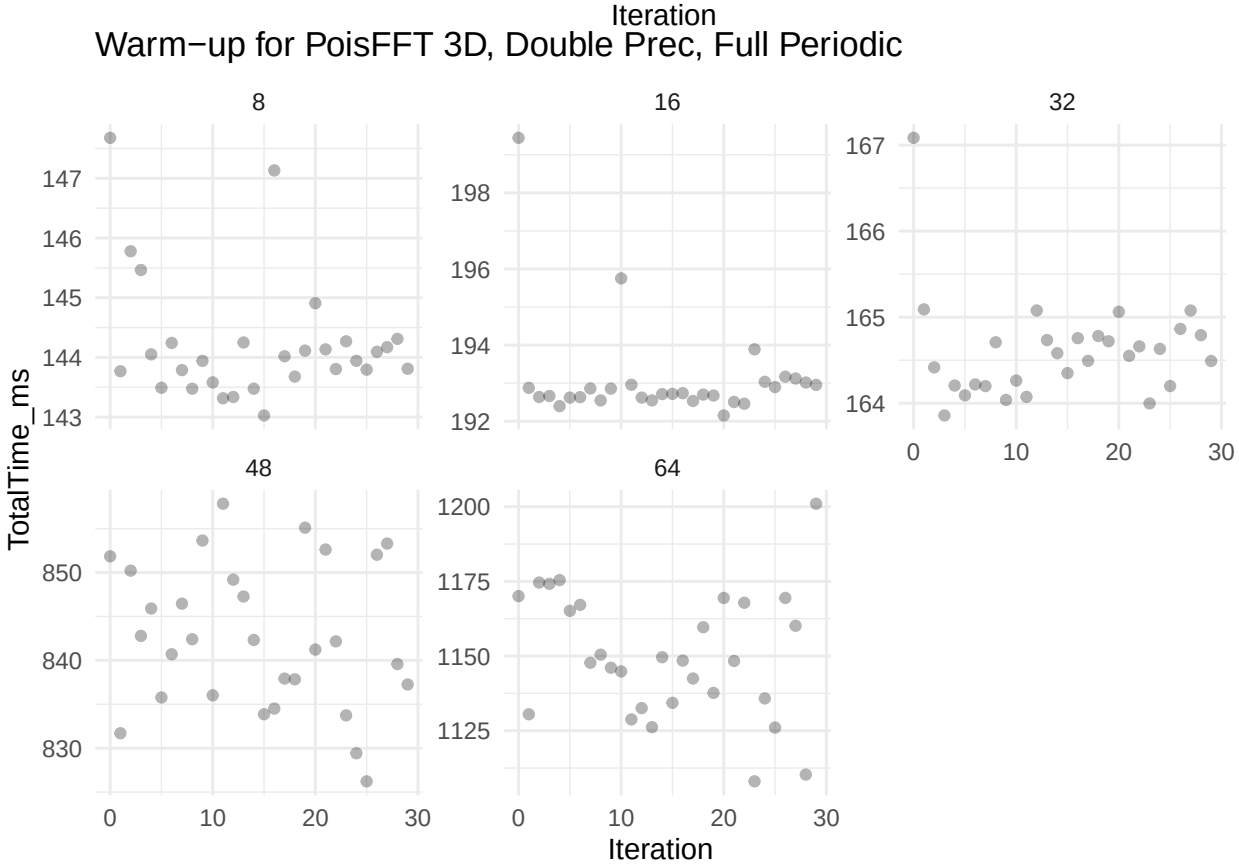

Warm−up for poisson−solver 3D, Double Prec, Full Periodic, CPU API
1
2
4
6
8
TotalTime_ms
Iteration

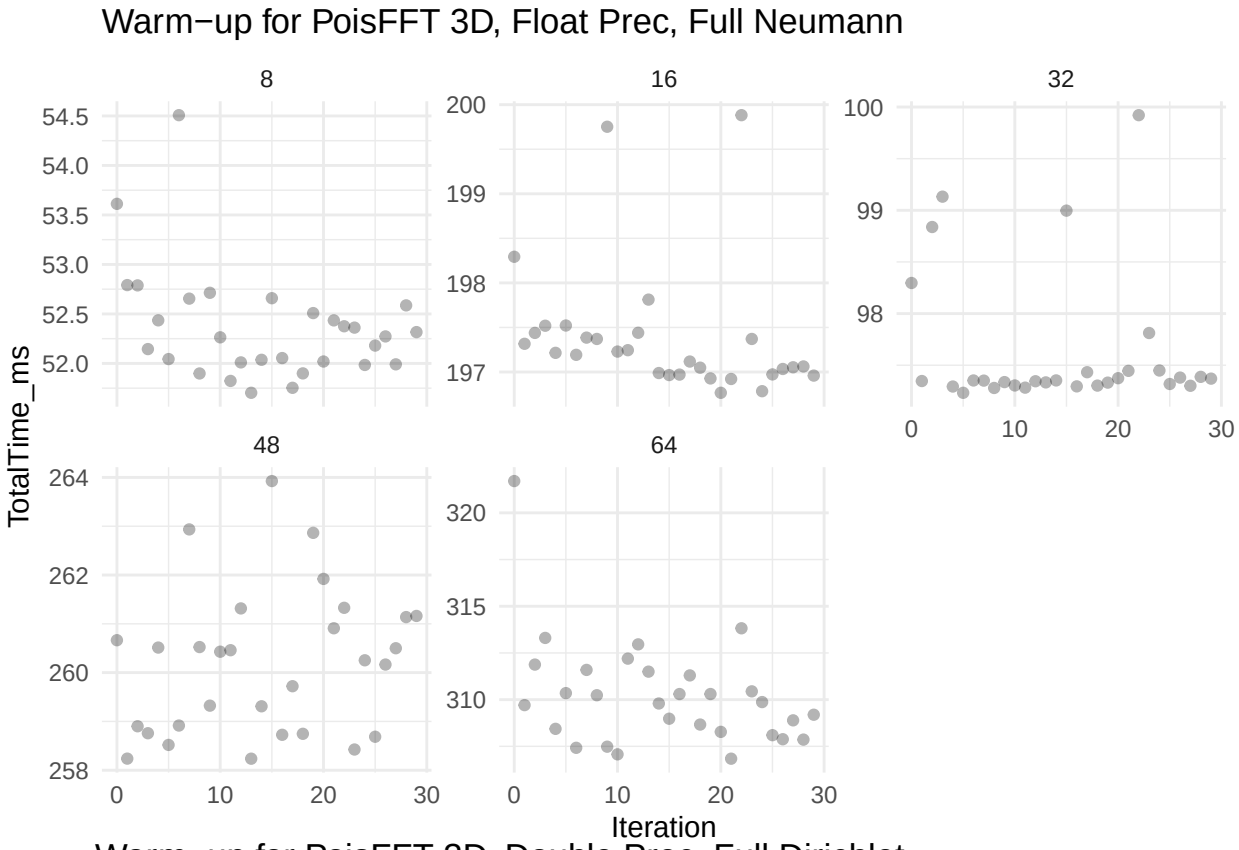

Warm−up for poisson−solver 3D, Float Prec, Full Neumann, CPU API
1
2
4
6
8
TotalTime_ms
Iteration

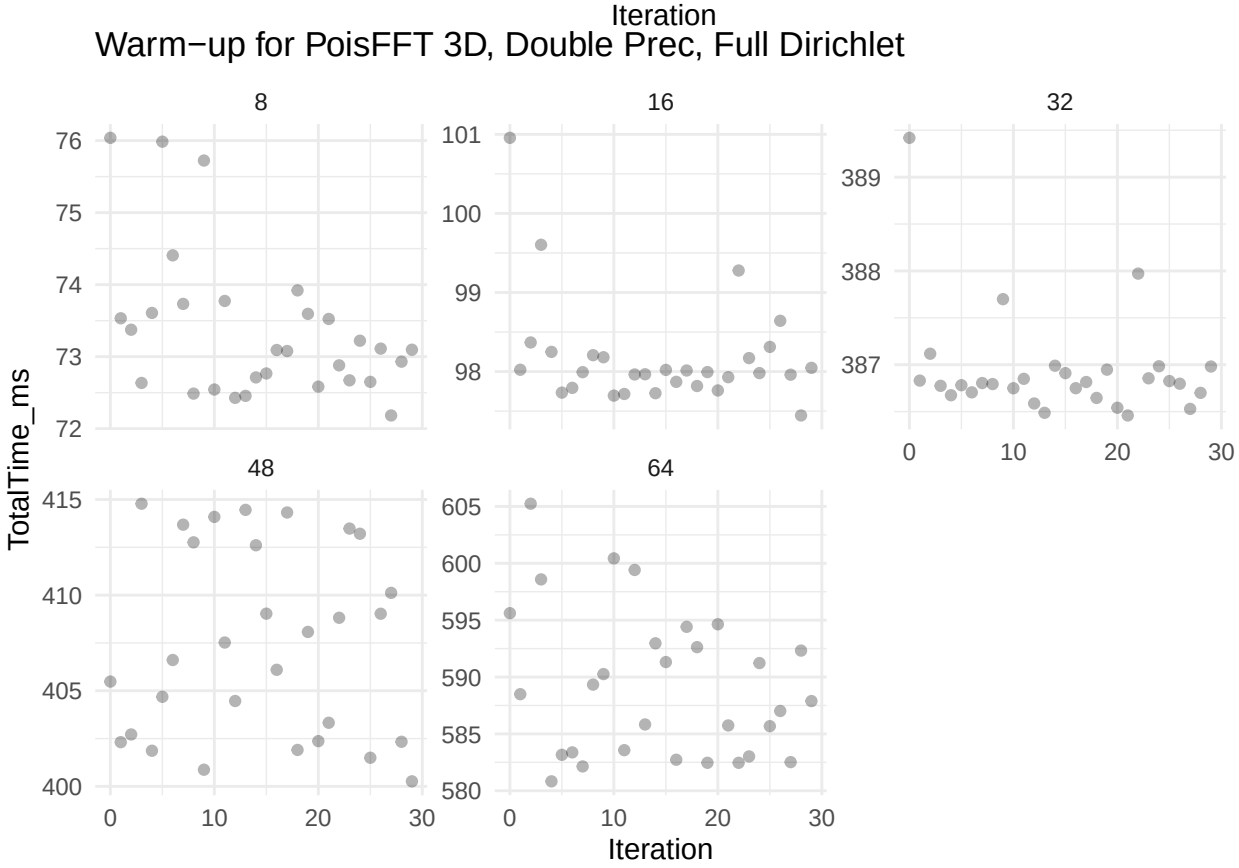

Warm−up for poisson−solver 3D, Double Prec, Full Dirichlet, GPU API
1
2
4
6
8
TotalTime_ms
Iteration

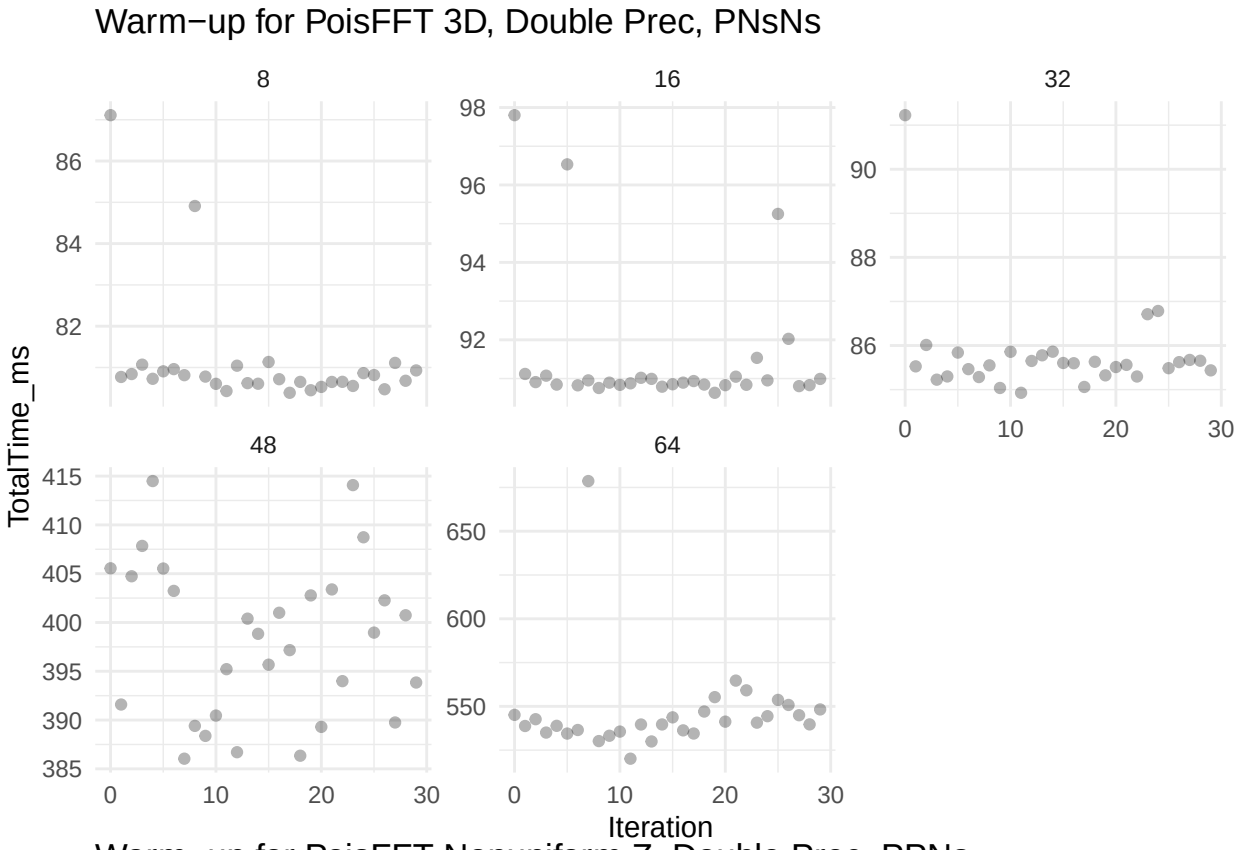

Warm−up for poisson−solver 3D, Double Prec, PNsNs, CPU API
1
2
4
6
8
TotalTime_ms
Iteration

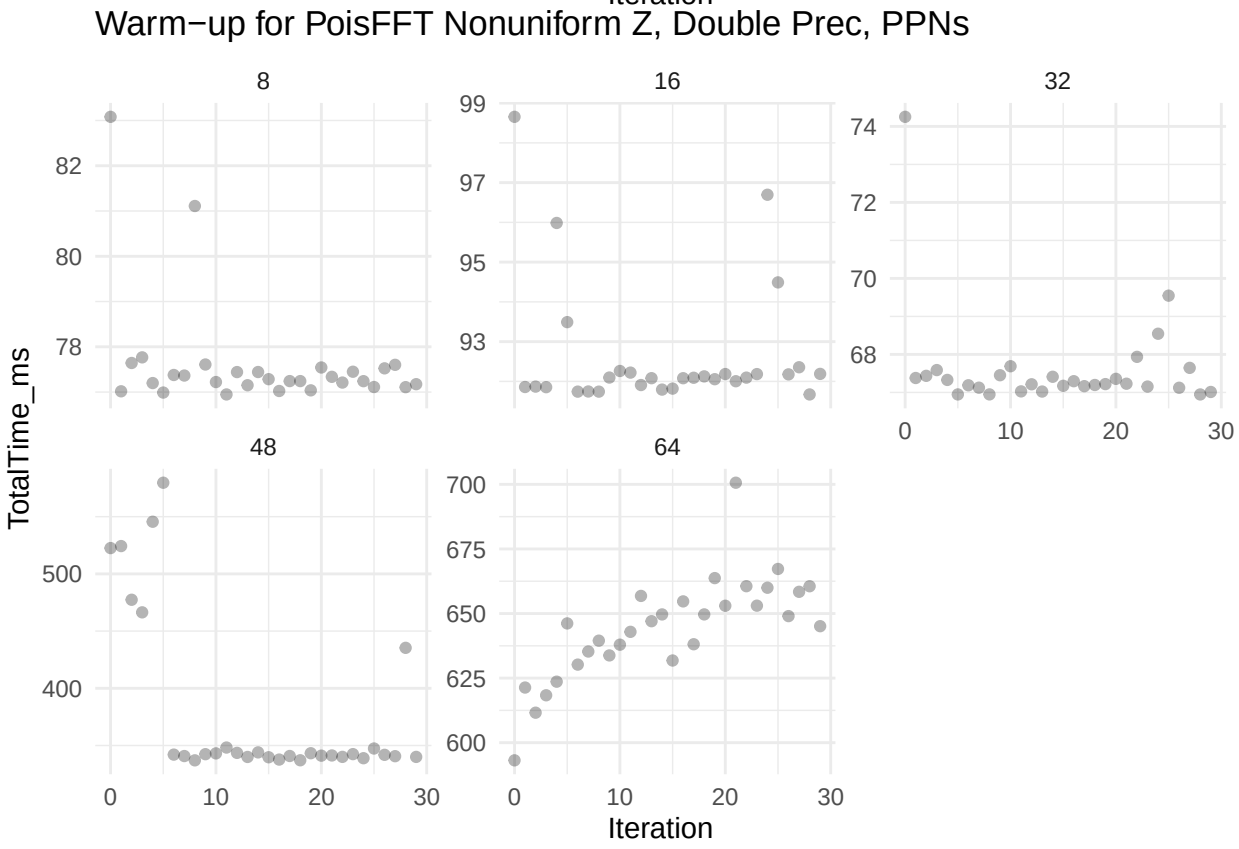

Warm−up for poisson−solver Nonuniform Z, Double Prec, PPNs, CPU API
1
2
4
6
8
TotalTime_ms
Iteration


Warm−up for PoisFFT 3D, Double Prec, Full Periodic
8
16
32
48
64
TotalTime_ms
Iteration


Warm−up for PoisFFT 3D, Float Prec, Full Neumann
8
16
32
48
64
TotalTime_ms
Iteration


Warm−up for PoisFFT 3D, Double Prec, Full Dirichlet
8
16
32
48
64
TotalTime_ms
Iteration





Here, it appears that the measurements have at most 6 warm-up iterations. We again discard those along

with times greater than 3σ.

Strong Scaling

The plots can be found in `plots/distributed/strong_scaling`.

`poisson-solver` is faster than `PoisFFT` in all configurations. `poisson-solver GPU API` is faster than `CPU API` in every scenario and the factor by which it is faster correlates with local problem size. This is expected, as local problem shrinks, there is less data to be copied between the host and the device.

`poisson-solver` doesn't scale well when adding new nodes. From one to two nodes, this is somewhat expected as that addition includes inter-node communication. From two to three nodes, there seems to be a plateau. Both could result from the fact that new node addition includes only two `Volta` GPUs while the communication is carried over 10Gb Ethernet. It would be better to measure this on nodes with InfiniBand connection and acquire measurements for more than 3 nodes (or at least more GPUs on those few nodes). `PoisFFT` sometimes exhibits similar behavior. The reason for not always exhibiting the same behavior could be due to FFT libraries transferring different amounts of data due to different packing strategies.

Weak Scaling

The plots can be found in `plots/distributed/weak_scaling`.

We can see `poisson-solver` outperforming `PoisFFT` in every configuration. Once again, `poisson-solver GPU API` is faster than `CPU API`, this time by a constant factor (local problem size stays constant). The scaling appears to be $O(log(N))$ (previously reported scaling for `PoisFFT` [1]) for `poisson-solver` but with different logarithm bases for $<= 1$ node and $>= 1$ nodes. `PoisFFT` scaling appears to be $O(log(N))$ with $<= 1$ node but chaotic above that.

Bar Plot Analysis

The plots can be found in `plots/distributed/bars`. The plots are made from the strong scaling data with three node utilization. Beware that full periodic configuration merges reshape and FFT times together because both are executed by an external library in the same FFT call.

The clear trend is that reshapes take majority of the time. This could be again due to the Ethernet connection between nodes. Buffer copy times don't pose as significant bottleneck here.

Speedups

The speedups can be found in `tables/`.

These speedups reach `~4.6x` for `CPU API` and `~5x` for `GPU API` in 3 node utilization. However for most BCs, they are in the range of `1.2-1.5x`.

Conclusion for Distributed Solvers

For distributed solvers `poisson-solver` improves `PoisFFT` in every scenario. The speedups aren't as impressive as for local solvers, this is largely due to the fact that this problem is communication-bound on the current cluster `volta` setup.

From the measurements it is apparent that simply migrating the code onto a GPU provides some performance improvements. Revisiting this benchmark on a setup with InifiniBand connection could yield further insight into the bottlenecks of distributed solvers.

ELMM

We will evaluate `ELMM` using the `channel` example. The GPU version of `ELMM` can be found here.

For the local setup, we will use `bw01` node, and for the distributed `volta[02,03,05]`. The grid size will be increased to 512 in each direction for the local setup and to 256 in each direction for distributed setup. The end time will be lowered to `0.1` for the local setup and to `0.5` for distributed setup.

`ELMM` and `gpu-elmm` will be built using the `make_release` and `make_mpi_release` scripts.

The boundary conditions will be:

1. All `periodic`,
2. `periodic` on all sides but top and bottom which will be `noslip`,
3. all `noslip`.

Local Solvers

You can find the results in `results/elmm-48-cores.txt`.

The speedups of full periodic and full noslip (NeumannStag) are similar to those observed in the benchmarks.

However PPNs speedup from `ELMM` is `~1.25x` while the benchmarks report `~2.15x`. We think this could be due to cache conflict misses but haven't investigated enough to confirm this. We only confirmed that with ghost cells present for `PoisFFT` in the benchmark the speedup really is `~1.25x`.

We have updated the benchmarks to contain those ghost cells and plan to remeasure the data on 14.9.2026 when the cluster maintenance ends.

Distributed Solvers

You can find the results in `results/elmm-distributed.txt`.

The speedups are generally a bit lower than what was measured. We didn't expect perfect agreement as the speedup table was constructed using 512^3 grid while this uses 256^3 grid, however it is alarming that the speedups are consistently lower. This could be due to the issue described in the local solvers section above but the network communication times mitigate the problem to not be as apparent here as in the local case.

Unfortunately most of `ELMM` is a CPU code which is significantly less parallelized when using the GPU setup (each process has its own GPU). This results in `gpu-elmm` being much slower than its CPU counterpart even though the solver itself is faster.

This could be solved by either porting all of `ELMM` to GPU or at least implementing hybrid OpenMP + MPI parallelization for `ELMM` which would enable us to launch the job with less processes but parallelize them using a GPU as well as CPU threads.

References

1. FUKA, V. PoisFFT – a free parallel fast poisson solver. *Applied Mathematics and Computation*. Online. 2015. Vol. 267, p. 356–364. DOI https://doi.org/10.1016/j.amc.2015.03.011.